大模型入门教材

训练一个可靠的学习助手

从语言预测出发经过示范与反馈走向检索和工具

问题驱动的连续叙事九个部分三十八节

面向具备基础编程能力的大模型初学者

图解修订版 2026 年 10 月 16 日

李孙博闻北京理工大学

写在开始之前

这本教材邀请我们共同完成一件事：把一个善于续写文本的大语言模型（Large Language Model，LLM），逐步变成能讲题、能查资料、能使用工具，也能承认信息不足的学习助手。我们暂时叫它“小助”。名字只是为了叙述方便；我们不会把它的生成行为当成人的意识、意图或成长经历。

我们会保留一条清楚的故事线。起初，小助连“请回答问题”都不总能做到。我们用示范教它回答，它开始像一个助手，却会在续训新课程资料后丢掉原来的讲解习惯。我们让它多试几次，发现偶尔能答对，于是尝试筛选和强化学习。分数开始上涨，我们又发现验证集没有进步。最后，我们把检索、工具和评估接入系统，才意识到：可靠的助手来自模型、数据、训练与应用共同构成的闭环。

我们默认自己会写基本程序，知道数组和函数的含义。正文首次引入术语时给出中文名、英文全称与常用缩写，后文沿用；模型名和论文题名保留原名，不另造全称。遇到概率、向量、梯度时，正文会就地补上够用的数学。初读时，我们先追踪“问题为什么出现、证据支持什么、下一步如何验证”；第二遍再推公式和复现实验。全书按九部分展开，适合约八到十二周的学习，也可以按课堂节奏拆分。每个编号小节都包含完整讲解，以及关键论文与补充材料的阅读入口。

书中“小助”的回答、行为和数值案例均为教学构造，用来练习推理与诊断，不是论文复现结果。实际实验需要记录模型版本、数据划分、模板、随机种子、计算预算与解码配置。

任何方法都不保证在所有模型和任务上有效；我们关心的是如何辨认有效条件。本教材尽可能不堆砌概念和知识点逐一介绍，而是尽可能拉通每个部分，希望读者能够自然地理解从上一步到下一步的逻辑关系，这样有助于理解整个大模型体系。本教材省略了许多技术细节，这些技术细节是成为大模型专家不可或缺的一部分，如果读者立志成为大模型领域的专家，建议在学习本教材的同时或未来能够自主广泛学习，打好基础。

本材料一到三章希望构建一个对大模型的基本理解，包括 tokenizer，词嵌入，句子嵌入，并且把大模型的行为理解为 next token prediction 的条件分布。四到六章串讲现代大模型全部训练方法。后续章节则尽量对大模型各个方向做一个入门性质的学习。

实际上，在当前的科研甚至工业研发场景下，本教材所提到的许多知识可能永远都用不上，例如一名后训练算法工程师通常只需要掌握数据合成、后训练方法（最基础的 SFT 和 RL）就可以完成大部分工作，甚至不需要关心词表、嵌入层等模型架构的知识，一名大模型工程师真正重要的能力在于碰到问题时能够有思路去摸索和解决问题，这就要求我们有尽可能准确和发散的思维，有科学可复用的方法论指导实践。

如果学有余力，强烈建议读者自学 Hugging Face 大语言模型课程（Hugging Face LLM Course）与 Stanford CS336《从零构建语言模型》（Language Modeling from Scratch）的官方材料，分别补充工程操作与从零构建细节。

目录

写在开始之前 **i**

第一章 定义一个值得训练的助手 **1**

1.1 任务定义与评估从一次失败开始 1

1.2 训练阶段语言学习类比能解释什么 2

第二章 语言模型与文本预测 **4**

2.1 分词器 Tokenizer 模型看到的是什么 4

2.2 下一个词元预测 NTP 从损失到信息压缩 7

2.3 Transformer 一次前向计算如何完成 10

2.4 预训练与缩放定律怎样分配训练资源 14

2.5 幻觉与前缀错位为什么会连贯地写错 15

第三章 嵌入表示与语义检索 **16**

3.1 从关键词匹配到语义向量为什么文字不同也能找到答案 16

3.2 Bert 模型怎样把相关文本训练得更近 17

3.3 从语义向量到真正的检索为什么不应直接淘汰关键词检索 20

第四章 监督微调与续训退化 **27**

4.1 监督微调 SFT 用示范塑造助手行为 27

4.2 SFT 数据工程模板与监督位置 28

4.3 续训退化新任务变好为何旧能力变差 29

4.4 遗忘诊断用对照实验选择修复方法 30

第五章 采样与强化学习 **32**

5.1 解码与采样多次尝试能发现什么 32

5.2 拒绝采样微调 RFT 把成功尝试变成示范 33

5.3 人类反馈强化学习 RLHF 从偏好到策略更新 34

5.4 组相对策略优化 GRPO 同题比较如何产生信号 36

5.5 RL 泛化诊断训练涨分而验证不涨 37

5.6 RL 训练诊断从数据与判题检查到补回 SFT 38

第六章 偏好优化与同策略蒸馏 40

6.1 直接偏好优化 DPO 为什么可以绕过奖励模型 40
6.2 离线与在线偏好优化数据如何跟上策略 42
6.3 同策略蒸馏 OPD 让老师反馈学生轨迹 43
6.4 后训练方法比较依据信号与分布作选择 44

第七章 推理策略与推理时计算 46

7.1 思维链 CoT 与推理时计算 46
7.2 推理模型从示范和结果反馈到蒸馏 47
7.3 KV 缓存与推理服务长上下文的资源成本 48

第八章 检索工具与系统评估 50

8.1 检索增强生成 RAG 让回答获得外部依据 50
8.2 RAG 诊断沿证据链定位失败 51
8.3 提示与智能体从表达要求到工具调用 52
8.4 分层评估怎样证明系统真的变好 53
8.5 实践路线六次连接全书的实验 54
8.6 研究闭环从失败证据决定下一步 55

第九章 可解释性入门 57

9.1 可解释性的证据看到了什么与证明了什么 57
9.2 线性探针表示中能读出什么 58
9.3 激活修补用干预检验计算路径 58
9.4 叠加表示与稀疏自编码器一个神经元未必对应一个概念 60
9.5 最小解释实验从一个问题走到可复核的结论 61

附录术语与阅读方法 63

关键术语与边界 63

第一章　定义一个值得训练的助手

1.1　任务定义与评估从一次失败开始

我们把一道题交给小助：“请解方程 3(a + 2) = 21，并解释每一步。”我们期待它先除以 3，再减去 2，得到 a = 5。它却接着写：“这是初中代数教材中的一道基础题，下面还有几道同类练习。”句子很通顺，也与题目有关，但它没有完成任务。

我们容易把这次失败归结为“模型不够聪明”。可如果把提示改成一段问答示例，再把同一道题放在末尾，它也许就能给出正确答案。此时，算力和参数都没变，变化的是上下文。这提醒我们，至少要区分三个问题：模型能否表示所需知识；在当前提示下会生成什么；模型是否把这种生成转成了用户需要的行为。

基座语言模型学习的是文本延续规律。真实语料里，问题后面可能跟答案，也可能跟题目分析、引用、另一道题，甚至空白。我们给了一个任务，小助并不会自动地将这个任务作为唯一目标，因为它还不知道问题和回答之间的区别，哪里是问题的开始，哪里是问题的结束，哪里是回答的开始，等等。

在训练之前，我们通常先写一份小型验收约定。我们选出四类需求：讲清基础题；回答课程资料问题；在缺少资料时说不确定；通过计算器处理容易算错的数值。我们把“正确率”和“讲解质量”分开，把“引用存在”和“引用支持结论”分开。一个回答可以算对但讲错，也可以格式漂亮却没有依据。

我们通常准备三份数据。训练集用来更新参数；验证集用来选提示、超参和检查点；最终测试集留到方案确定后使用。以后若反复根据同一份验证集修改方案，它事实上已经参与了开发，不能继续被当成完全独立的泛化证明。同一道母题的改写也应成组划分，避免训练和评估只是换了姓名与数字。当然，这是传统机器学习的思路，许多大模型的工程实践中基本没有验证集，研发人员直接合成训练数据并且在测试集上测试效果并且根据结果反馈调整训练集，尽管这是不够严谨的。

从这一步起，我们把“效果好像变好了”细化为成可核验的问题：哪类任务改善了，哪类退步了，在相同预算下是否仍成立？这份最初的评估约定，会在监督微调（Supervised Fine-Tuning，SFT）续训和强化学习（Reinforcement Learning，RL）过程中遇到麻烦时成为我们的定位工具。SFT 用示范回答训练模型，RL 则依据反馈优化模型的尝试；我们稍后会逐步展开。

探索问题如何解释对于预训练模型：“我们容易把这次失败归结为“模型不够聪明”。可如果把提示

改成一段问答示例，再把同一道题放在末尾，它也许就能给出正确答案。”这个现象。

本节相关资料

• Brown 等，2020，Language Models are Few-Shot Learners。原文链接 。阅读重点：同一个模型为什么能通过上下文示例切换任务。

1.2　训练阶段语言学习类比能解释什么

为了理解接下来发生的事，我们先想象一个熟悉的过程。婴儿长期听到周围的人说话，逐渐熟悉声音与表达；后来在互动中练习表达，得到示范和反馈。这能帮助我们建立一个入口：大量接触语言、学习具体表达、再依据结果调整，是彼此衔接但不相同的学习环节。

我们可以把预训练类比为广泛接触语言，把监督微调类比为有人示范“面对这个问题，可以这样回答”，把反馈学习类比为比较不同回答是否合适。这个类比让抽象的训练阶段有了生活背景，但仍然有一点区别：模型不会经历人类婴儿的发育过程。

婴儿有身体、感知、共同注意和社会互动。文本语言模型的预训练通常是在已有序列上预测下一个词元（token，即分词后的基本处理单元），并通过误差更新参数。注意，当我们提到大模型的行为时，经常指代的就是模型的“分布”。它在预训练时已经反复“尝试预测”，并非只听不说；SFT 也常使用同一种逐 token 交叉熵（cross-entropy）损失，并不是第一次获得生成能力。两者的重要区别在于数据的组织、监督的位置和我们希望强化的行为分布[1]。

预训练已经初步确定了 next token 分布，所以后训练叫做微调。不过因为注意力机制的存在，实际分布由参数和上下文共同决定，所以对于一个问题，后续解答时 next token 分布不仅由问题本身决定，也跟其他上下文有关，比如问题前面可能还有一个解题指导，有指导地解题和无指导地解题难度肯定是不一样的，所以分布也有差异。全量后训练通常会训练大量数据，这样也可以比较明显改变模型的分布，这种规模的训练就不再是“微调”了。我们还可以做小规模后训练，比如在别人已经训练好的模型上继续用比较少的数据做 SFT 或者 RL 训练，这种小规模后训练的能力范围受训练起点、数据、优化与预算限制，通常很难大幅度调整模型的行为，甚至可能导致模型的通用能力收到影响。类似于一个人在大学系统性学习了四年中国哲学，后来自学了几个月西方哲学，那么西哲主要是做了一点思想补充和调整，其主干思维可能还是以中哲为主。

不妨想象小助对一道题能给出三种回答。训练前，它有时直接给答案，有时续写题目，有时解释步骤。SFT 提高给出高质量解题流程的概率；RL 依据奖励重新分配不同尝试的概率，比如同一个题目可能有好几种解题方案，可以通过给更好的解题方案更高的奖励来使得模型更倾向于生成这种更好的解题方案，这是一种偏好调整。若它从未形成某个关键运算过程，单靠稀疏奖励去生成完整解法可能很困难；若它已经偶尔做对，强化学习的奖励就可能帮助它更稳定地选到有效路径，比如解题 10 次，只对了 2 次，我们奖励这两次形成的解题轨迹，模型就会在这类问题上更倾向于走这条轨迹。

因此，这本书中的“学会”和“知道”都是方便交流的简写。落实到实验，我们总会追问：在哪些输

[1]理解大模型的输出（行为）是基于概率分布的，即根据上下文，从下一个 token 的概率分布中采样

入、怎样的解码条件和多少预算下，模型表现出了什么行为？有了这层约定，我们就能借助故事建立直觉，同时用机制纠正直觉。

实践查资料或者实践看一下预训练数据和后训练数据有什么区别。

本节相关资料

Ouyang 等，2022，Training language models to follow instructions with human feedback，也就是 InstructGPT。原文链接 。阅读重点：如何利用人类反馈来优化大模型的行为。**该文章是人类偏好强化学习（RLHF）的里程碑论文之一，可以提前阅读，也可以在学到强化学习时阅读**。

第二章　语言模型与文本预测

2.1　分词器 Tokenizer 模型看到的是什么

我们决定先观察小助的输入。屏幕上是一段中文题目，进入模型的却是一串整数。这些整数不是天然对应每个汉字，也不一定对应一个完整单词；它们是分词器（tokenizer）按词表规则把文本切分后得到的词元标识符（token identifier，token ID）；后文沿用 token 和 token ID。

我们用一个极小例子理解子词分词。若词表保存完整英文单词，遇到从未见过的复合词就很难处理；若只保存字符，序列又会变长。字节对编码（Byte Pair Encoding，BPE）一类方法从小单元出发，逐步合并常见相邻片段，在词表大小与序列长度之间折中。实际 tokenizer 还有字节回退、预处理和特殊标记等差异。

一个经典的例子：我们输入 strawberry，想让小助数出字母 r 的个数。人眼可以直接扫描字符；模型可能看到几个子词片段（比如 staw 和 berry 两个 token，而不是 s t r a w…这样把每个字母都切分开），模型就需要把片段重新关联到字符，再执行计数，比如模型可能知道 berry 这个 token 和字母 r 是有关系的，然后就会思考 berry 这个 token 由哪几个字母 token 组成，模型能够做到这一点，可能是因为预训练数据中有一些关于单词拼写的语料，或者后训练中有专项训练。分词让任务的表示过程更绕，许多模型都会数错，但不能断言模型一定数错：模型可以通过训练、内部计算或外部工具完成任务。这个例子说明的是输入文本的粒度与模型理解的任务粒度是不同的。

对话还有一层常被忽略的编码。我们在界面里看见系统（system）、用户（user）、助手（assistant）三种角色，模型实际接收的通常是由对话模板（chat template）拼成的序列，其中含有角色边界、轮次结束和生成起点等标记。角色名称可能是普通文本与特殊 token 的组合，不是所有模板都把每个角色名设为一个特殊 token。建议在 AI 的帮助下看一些实际案例，理解模型如何切分或组合文本为 token。

当我们问问题的时候，大模型看到的内容及其完整回答里面有许多标记，自然语言夹杂在这些标记中间，这些标记是为了方便机器区分不同的文本对话。我们问：“洗车店在 50 米外，我要洗车，是开车去还是步行过去”，许多模型会给出错误的回答，比如走过去，实际上洗车就必须开车去。下面是一个模型所看到的完整序列：

我们拆解一下具体过程：

<|im_start|> user\n洗车店在 50 米外，我要洗车，是开车去还是步行过去 <|im_end|>\n<|im_start|> assistant\n<think> 用户问的是：洗车店在 50 米外，要洗车，是开车去还是步行过去。\n\n这是一个简单的

生活常识/决策问题。50 米非常近，大概就是步行几十秒的距离。开车反而可能更麻烦（找停车位、倒车入库等），而且 50 米开过去可能一个路口都没到就到了。\n\n我应该给出简短、明确的建议：步行过去。理由：50 米很近，步行约一分钟，开车反而费事（找车位、倒车等）。</think>50 米太近了，** 步行过去 ** 最省事。\n\n- 步行也就 ** 几十秒到一分钟 **，一脚油门一踩就过了，根本不值得专门发动车子。\n- 开车过去还得找停车位、倒车入库，搞不好比走路还费劲。\n- 还省了点油耗和停车费。\n\n所以锁好车，走过去就行，到了直接把钥匙交给店员。<|im_end|>

首先我们发送了一个问题，那么此时一定是对话的开始，所以自动在开头以 “<|im_start|> user\n” 开始，表示用户问题开始，然后拼上我们的问题，在问题结束以后自动拼上 “<|im_end|>\n”，表示用户问题结束，接下来需要加上标记让模型知道轮到它生成，类似前面的标记，拼上 “<|im_start|> assistant\n”，并且我们要求模型必须先思考再回答，所以加上 “<think>”，那么模型在自己生成之前，看到的内容就是前面这些，接下来模型就自己生成一段思考内容，模型自己决定在哪里结束，在适当的位置自己拼上 “</think>” 结束思考，接下来就是面向用户的总结回答，回答结束，模型自己拼上 “<|im_end|>”，这就是一个完整的生成过程。

下面是一个比较清晰的图例，拆分了上面的原始文本：



为什么这些标记有效，以及为什么大模型认识这些标记？模型在预训练的时候是不会见到这些标记的，但是在后训练的时候会接受大量带有这些标记的数据训练，训练之后，模型的行为分布就会变成这样，比如，不管上下文是什么，在 <think> 之后一定是思考的内容，思考完毕一定要生成一个 </think>\n\n，然后才是给用户的回答，所以当我们要求模型进行思考时，就在模型开始推理之前拼

上 <think>，如果我们问的是一个很简单的问题，不希望模型思考，只是要模型快速回答，那么就在开始推理之前拼上 <think></think>\n\n，模型就知道现在不需要思考，直接给出最终回答。我们与大模型对话的时候有一个思考开关，这个开关控制的就是这个 think 标签如何拼接在模型回答的开头。但是这个行为也是训练的概率分布，既然是概率分布就会有小概率事件，许多模型可能在 1% 的案例中缺少 </think> 标签或者出现其他格式错误，可能进一步造成前端显示错误。

假如训练时回答前有起始标记，推理时却漏掉起始标记，小助可能把用户内容（问题）继续写下去。假如停止标记 <|im_end|> 配置错了，它可能回答完以后继续模拟用户提问，因为他不知道回答结束了，会继续写下去（因为预训练的时候问答语料可能就是这样的，在答案后面跟着下一个问题）。**还有思考标记 <think> 也可能出问题，在真实工程中，有些模型的 <think> 后面一般还必须跟着两个回车（\n\n），如果我们的模板错了，没有这两个回车，模型的表现会打折扣，原本在一个榜单上可以做到 90 分，但是因为模板错误，少了两个回车，可能就只有 85 分**（这种分数不一致的情况称为精度错误，其原因可能是模板错，可能是模型配置错，甚至可能和推理框架的 bug 有关）。一般来说，think 和工具等标记可能由模型自己生成，其他的标记可以由脚本自动生成，比如在模型的权重目录中有一个 chat_template.jinja 文件来自动处理标记，这个功能也可能集成到其他的配置文件中，建议读者在 huggingface 或者 modelscope 网站中找到一些常用的开源模型，看看这些模型的文件构成。

序列长度还决定成本。同一份资料在不同 tokenizer 下可能占用不同数量的 token，因为分词规则不同；长序列会影响训练显存、截断位置、推理延迟和可用上下文。因此，“字符数” 与 “token 数” 不能互换，模型的序列长度计数是基于 token 的。

我们还要区分 “切分有错” 和 “某个 token 难以生成”。MiniMax 在 2026 年公布的 “马嘉祺” 案例中，人们发现 Minimax M3 模型知道 “马嘉祺” 是谁，但是无法输出 “马嘉祺” 三个字。他们的研究案例提醒我们：一个词生成不出来，不足以证明 tokenizer 损坏。MiniMax 的研究者对这个问题进行了非常有参考价值的探索研究并且写成博客[1]。

实践在 OpenAI 的分词器网站[2]里面尝试各种文本，看看大模型是如何分词的。

本节相关资料

Touvron 等，2023，Llama 2 Open Foundation and Fine-Tuned Chat Models。原文链接 。阅读重点：对话训练与模型使用约定如何共同影响行为。

（建议阅读）MiniMax Research，2026，为什么 MiniMax 大模型不认识 “马嘉祺” 稀疏 Token 遗忘排查记录研究博客。原文链接 。阅读重点：如何先排除分词不一致，再检查后训练变化；与第 4.3 节对照阅读，搞清楚这个问题还需要对大模型结构有了解，比如 LM head。

[1] https://www.minimax.cn/blog/sparse-token-forgetting-investigation

[2] https://platform.openai.com/tokenizer?utm_source=chatgpt.com

2.2　下一个词元预测 NTP 从损失到信息压缩

本节内容对初学者来说可能比较抽象，并且暂时也不是需要彻底理解的内容，或者，观看这段 信息压缩 视频解说可平替本节学习。

现在我们把小助暂时简化成一个函数：输入此前的 token，输出词表上所有候选 token 的概率。对 “法国的首都是” 这个前缀，它可能给 “巴黎” 相关的首个 token 较高概率，也给其他延续保留概率。生成时，我们从这些概率中选一个 token，接回前缀，再重复同样的过程。

这就是下一个词元预测（Next Token Prediction，NTP）。给定此前的 token，模型对下一个位置给出一个条件概率分布：

$$p_\theta\left(x_t \mid x_{<t}\right)=p_\theta\left(x_t \mid x_1, \ldots, x_{t-1}\right)$$

这里 θ 表示模型参数，t 表示位置，x 的下标表示序列中的 token，x 的下标小于 t 表示此前的完整前缀。对词表中所有可能的下一个 token，这些概率加起来为 1。我们不是先生成一句完整答案再给它评分，而是一步步预测。

若整段文本有 T 个 token，把这些条件概率相乘就得到序列概率。按照概率链式法则，自回归分解写为：

$$p_\theta\left(x_{1:T}\right)=\prod_{t=1}^{T} p_\theta\left(x_t \mid x_{<t}\right)$$

上式中，x 的下标 1 到 T 表示整段序列，T 是序列长度，乘积号表示把各位置的条件概率相乘。每个位置都贡献一个预测任务，因此一段普通文本就能产生大量监督信号，不必另外给每个句子标上 “语法课” 或 “事实课”。

怎样惩罚预测错误？我们取真实下一个 token 的概率，再取负对数。给正确 token 的概率越低，损失越大。若概率从 0.1 提高到 0.5，单个位置的负对数损失会从约 2.30 降到约 0.69。训练把大量位置的损失加总或平均，再用梯度更新参数。把长度为 T 的序列在各位置的负对数概率取平均，NTP 损失写为：

$$L_{NTP}=-\frac{1}{T}\sum_{t=1}^{T}\log p_\theta\left(x_t \mid x_{<t}\right)$$

这里 log 使用自然对数，损失的单位为 nat（奈特，自然对数的信息单位）；负号使低概率对应较大损失，前面的 1 除以 T 用来平均各位置。

为什么这个简单目标会与知识和推理有关？如果文本里出现 “把 21 平均分成 3 份，每份是”，准确预测后续往往依赖算术关系；如果出现人物代词，预测可能依赖前文指代。模型可以利用词频捷径，也可能形成更可迁移的内部表示。目标并不逐条规定它必须怎样表示世界，我们要用新任务检验它究竟学到了什么。

在分布层面，交叉熵可以拆成真实数据分布的熵与一个库尔贝克与莱布勒散度（Kullback–Leibler Divergence，KL 散度）。真实分布不随模型参数改变，因此最小化理想的总体交叉熵，等价于最小化数据分布到模型分布的正向 KL。有限训练集上的经验损失只是这个总体目标的估计，不能保证覆盖未见过的输入。设 p 是真实数据分布、q 是模型分布，交叉熵与 KL 的关系具体写为：

$$H(p,q) = H(p) + D_{KL}\left(p \| q\right)$$

$H(p,q)$ 是交叉熵，$H(p)$ 是数据自身的熵，D 的下标 KL 表示从 p 到 q 的分布差异。p 固定时，H(p) 不随参数变化，因此优化交叉熵与优化这一 KL 项具有相同的方向。

我们已经知道，给真实 token 更高概率会降低损失。可是，为什么有人又把这说成“更会压缩”？先不急着看区间表，我们让小助承担一个具体任务：把课程文本存进文件，再从文件中逐字还原。这里讨论的是无损压缩（lossless compression），目标是在不丢失原序列的前提下减少比特数；写摘要、删掉重复句、只保留大意，都没有满足这个目标。

先明确我们究竟在省什么假设消息只含 A 和 B，两者各用一个二进制位表示，100 个符号就要 100 bit。真实文本通常先由字符编码表示成字节，基线并非“一字一 bit”；我们先用二符号信源，是为了把统计结构与文本编码的细节分开。压缩器最终存下的是比特流，不是一段更短但意思差不多的自然语言。

编码器看到了原消息，解码器没有看到。两端要约定符号顺序、概率模型和消息如何结束，解码器才能从比特流唯一恢复原消息。若必须把新词表、概率表或模型权重一并发送，这些也属于总成本；只报“正文编码用了多少位”会夸大节省量。图 2.1 先把这份通信约定画出来：

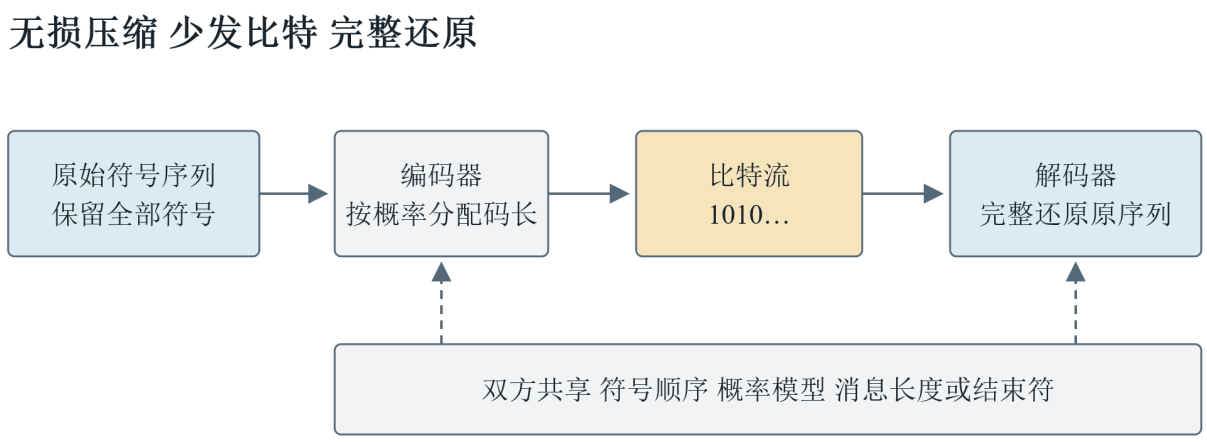


图 2.1 无损压缩的目标与两端约定。发送的是足以唯一定位原消息的比特流，接收端必须还原同一条符号序列。

为什么概率能够帮助省位数在可唯一解码的约束下，我们不能给每条消息都分配更短的码：短二进

制串的数量有限。统计压缩把较短的表示优先留给常见消息，让不常见的消息承担较长的表示，从而降低平均成本。因此，压缩器可能让个别输入变长，尤其是很短或接近随机的输入；无损压缩不承诺每一个文件都缩小。

例如，若符号独立产生且 A 的概率为 3/4、B 为 1/4，我们可以把许多符号一起编码，长消息的平均每符号码率能够接近 0.811 bit，而固定表示是每符号 1 bit。这里比较的是指定信源下的长期平均量；实际有限文件还有终止、对齐和元数据开销。若 A、B 各占一半且彼此独立，就没有这份频率偏斜可利用；如果它们有上下文依赖，仍可以由条件模型进一步利用。

算术编码怎样把概率变成比特算术编码（arithmetic coding）让编码器与解码器按相同概率划分区间。我们先从 $[0, 1)$ 出发，每读一个符号，就按概率切分当前区间，只保留实际符号对应的那一段。待整条消息处理完，编码器发送一段二进制前缀，只要它表示的整个小区间都落在目标区间内，解码器就能确定这条消息。

我们用固定概率走一遍：A 的概率为 0.75，B 为 0.25，消息为 ABA。第一次读 A，保留 $[0, 0.75)$；第二次读 B，保留当前区间最后四分之一，得到 $[0.5625, 0.75)$；第三次读 A，保留当前区间前四分之三，得到 $[0.5625, 0.703125)$。下面每行把上一行选中的区间放大，横轴的尺度随行改变：

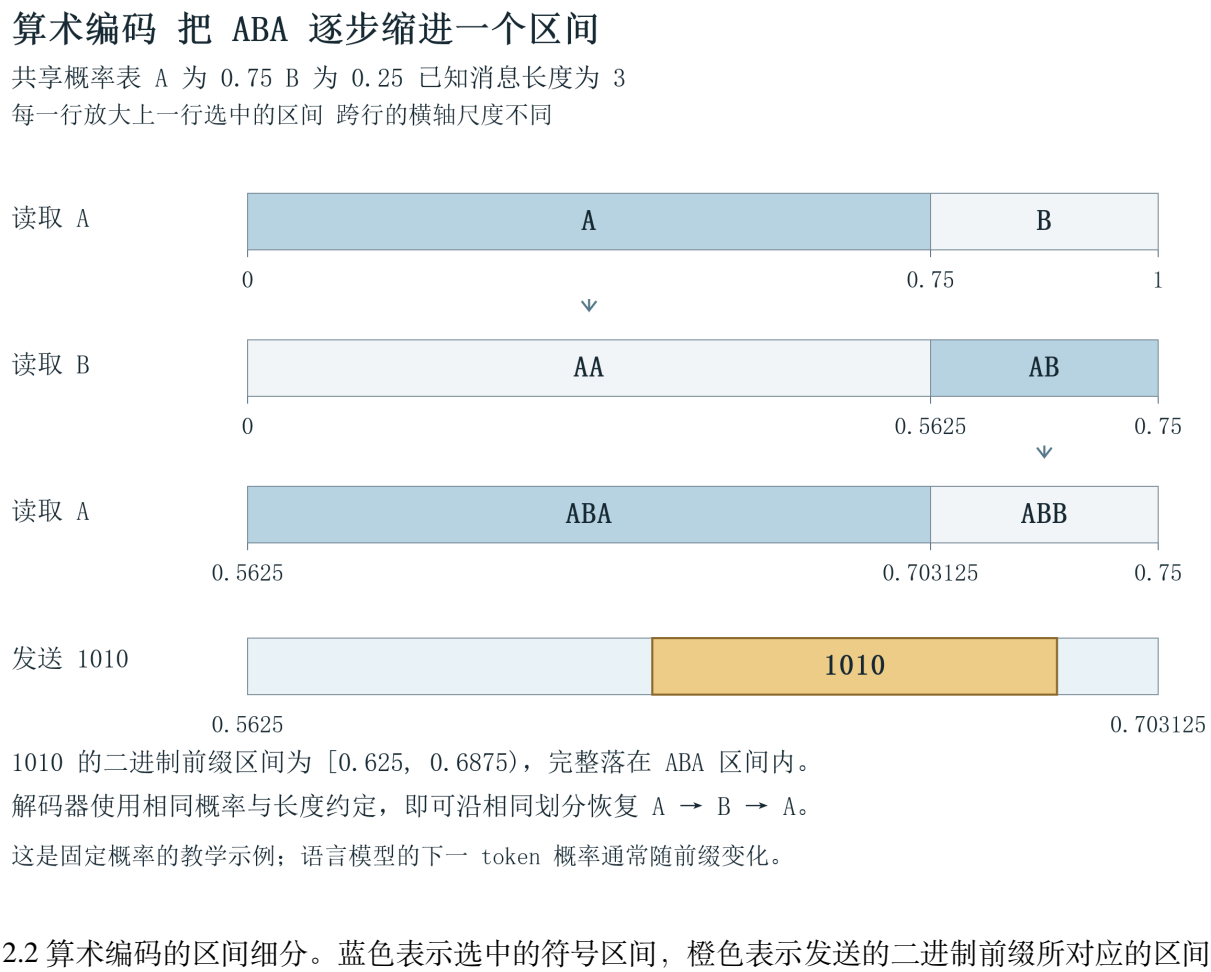


图 2.2 算术编码的区间细分。蓝色表示选中的符号区间，橙色表示发送的二进制前缀所对应的区间；跨行不能按图形长度比较绝对宽度。

1010 为什么足以还原 ABA 二进制小数前缀 0.1010 的后续位尚未确定，因此它代表的全部数值位于 [0.625, 0.6875)。这个区间完整落在 ABA 的最终区间里。解码器可以取其中一个数，例如 0.625：它位于第一层的 A 区间；继续按同一概率细分，位于第二层的 AB 区间；再细分，位于第三层的 ABA 区间。双方已约定长度为 3，于是停止；若长度未知，就还需要结束符或其他长度协议。实际实现还要处理有限精度和逐步输出比特，我们这里先追踪数学上的区间。

这个例子发送了 4 bit，甚至比二符号固定表示 ABA 所需的 3 bit 更长。这并不矛盾：我们用极短消息展示了可逆机制，尚未展示长期平均压缩收益。若丢掉长度与模型开销、只看漂亮的区间图，就会把 “能编码” 误当成 “这个样本一定更省”。

码长怎样回到训练损失每选一次符号，区间宽度就乘上该符号的条件概率，所以最终宽度等于整串消息的模型概率。m 个二进制前缀位对应宽度为 2 的负 m 次方的区间。为了在目标区间内找到一个可用前缀，所需位数便与概率的以 2 为底负对数相联系。用 q 表示双方共享的编码模型、T 表示消息长度，理想信息量写为：

$$l_q\left(x_{1:T}\right) = -\log_2 q\left(x_{1:T}\right) = -\sum_{t=1}^{T} \log_2 q\left(x_t \mid x_{<t}\right)$$

这里 ℓ 的单位是 bit，x 的下标 1 到 T 表示整串消息，右侧按位置相加；它是理想信息量，不是任意有限文件的最终整数位数。ABA 的概率是 0.75 × 0.25 × 0.75 = 9/64，理想信息量约为 2.83 bit，而图中实际选用了 4 位前缀。

现在，前面的交叉熵也有了操作含义。假设真实消息服从 p，而我们按 q 分配码长，那么以 2 为底计算的交叉熵是理想编码长度的平均值；数据自身的熵描述信源不确定性，正向 KL 描述使用不匹配模型额外支付的平均长度。在同一数据上，把真实 token 的预测概率提高，会让对应的理想编码长度降低。

回到语言模型，我们可以让编码器与解码器共享同一个已训练模型。两端从相同起始状态开始；解码出一个 token 后，把它接回前缀，再算出相同的下一步概率。因此，解码器不需要提前知道原文，也不需要靠采样 “猜” 出原文，而是用收到的编码位从概率区间中确定原 token。两端的分词器、参数、数值实现与结束约定必须兼容。

前面的训练损失使用自然对数，单位为 nat；这里使用以 2 为底的对数，单位为 bit，除以 ln 2 即可把 nat 换成 bit。这个联系说明 “预测更准为什么可能让编码更省”，却不表示常规推理会输出压缩文件，也不表示模型权重是训练文本的可逆压缩包，更不能仅据此断言模型已经理解每个事实或解决每道题。知识与推理能力仍要通过独立任务检验。

2.3 Transformer 一次前向计算如何完成

如果觉得本节讲解过于简单抽象，可以自行查找相关视频学习，重点理解注意力计算、注意力掩码和位置编码 ROPE。

知道目标还不够，我们还要知道小助怎样把前缀变成概率。最早的 n 元语法（n-gram）模型只统计有限窗口中的词组合，窗口一长，组合就迅速变稀疏。神经语言模型让上下文共享参数与表示，循环网络再把历史压进状态。变换器（Transformer）架构则让不同位置通过注意力直接交换信息，成为一条适合并行训练的路线。这个历史并非一条必然链，但能帮我们看见每种设计回应的困难。

我们先把 token ID 送入嵌入（embedding）层。它相当于一张可训练查找表：每个 ID 对应一个向量。向量本身没有天然的 “第七维就是语法” 含义；其意义来自训练后与其他参数的协同。位置编码提供顺序信息，否则模型难以区分 “甲帮助乙” 和 “乙帮助甲”。其中，旋转位置嵌入（Rotary Position Embedding，RoPE）由苏剑林等人在 RoFormer 工作中提出：它按位置旋转查询向量与键向量的表示，让注意力分数能够体现相对位置关系。RoPE 的 “相对” 性质不等于可以无条件外推到任意长度。

进入一层注意力时，我们把这一层输入的每个位置表示排成矩阵 X。这里的 X 不是原始 token ID，而是已经进入模型计算的向量。同一个 X 分别乘上三套可训练投影矩阵，得到查询（query，Q）、键（key，K）和值（value，V）。它们来自同一批位置，但承担不同计算角色，不能把 Q、K、V 理解成输入中的三种不同词。

我们把三套投影矩阵记为 W 的下标 Q、K、V，投影关系写为：

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V$$

若 X 有 n 行，每行具有相同的输入隐藏维度，则查询与键投影把它映射到 d_k 维，值投影映射到 d_v 维；所以 Q、K 都有 n 行、d_k 列，V 有 n 行、d_v 列。这里省略投影偏置，并先讨论一个注意力头。下标用于标识投影角色，不表示它们要对不同的原文分别分词。

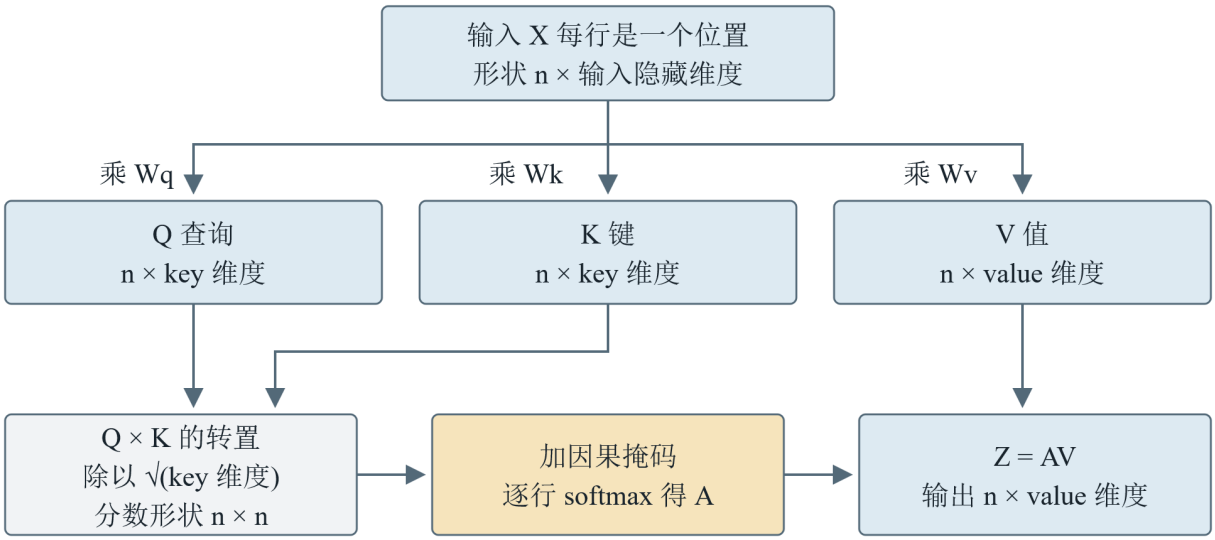
把所有位置的 query、key、value 分别排成矩阵 Q、K、V，缩放点积注意力的计算写为：

$$Attention(Q, K, V) = softmax\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)V$$

上标转置符号表示转置；softmax 对每个 query 的候选 key 分数归一化，再用权重汇总 V。因果掩码 M 在可见位置取 0，在禁止关注的位置用负无穷屏蔽。

其中 d 的下标 k 表示每个 key 的维度；缩放有助于控制点积的数值范围。Q 与 K 的点积是在对应维度相乘后求和，表达一种可训练的匹配分数。M 是掩码。对我们讨论的因果语言模型，未来位置被遮住；训练时即使整段文本同时进入计算，也不能偷看尚未预测的 token。多头注意力让不同子空间并行建立关系。注意力权重是计算的一部分，不能直接当作完整的因果解释。

我们可以把 query 理解为 “当前怎样匹配上下文”，key 理解为 “每个位置提供怎样的匹配线索”，value 是最终被汇集的信息。这个类比只是计算角色的直觉，向量维度不必对应人类能命名的概念。图 2.3 把分数、权重和被加权的信息分开：



Q 和 K 决定权重；真正被加权相加的是 V。这里画的是单个注意力头。

图 2.3 单头注意力的数据流。Q 与 K 产生位置之间的分数；掩码与 softmax 得到权重矩阵 A；再由 A 对 V 做加权和。n 是序列长度。

沿着一行真正算一次我们暂把 “小猫追球” 当作三个玩具 token，站在第二个位置 “追” 上。图 2.4 人为设定了该位置的 q_2，以及三个位置的 key 与 value；这些数只用于复算，并非从真实模型提取。q_2 与三个 key 的点积分别是 1、0、1，除以 $\sqrt{2}$ 后为约 0.707、0、0.707。

第二个位置只能读取第一、第二个位置，第三个位置是未来，因此先把第三个分数置为负无穷，再对这一行做 softmax。前两项得到约 0.670 和 0.330，第三项为 0；最后用这三个权重加权 value，而不是把 key 当作输出。图中结果是约 (1.340, 0.660)：

图 2.4 一个 query 的完整计算。softmax 在允许的位置之间归一化；禁止位置的权重为 0，不能先读未来信息再在输出端补救。

注意，“追” 位置更新后的表示可以用于预测后续 token，但当前这次注意力不能把未来 “球” 的内容拿来使用。训练时整段序列虽然并行送入模型，掩码仍保证每一行只有相应前缀可见。图 2.3 的 n × n 分数矩阵，就是同时对所有 query 做这样的行计算。

多头注意力把多套投影的结果拼接，再经过输出投影，回到能够与残差相加的隐藏维度。若采用 RoPE，位置相关旋转施加在 Q 与 K 上再计算分数，图 2.3 省略了这一环节以突出核心数据流；不能因此把位置编码理解成另一份 V。注意力权重也只是这次前向计算的系数，不自动等于因果解释，第九部分会专门检验这一点。

注意力后还有前馈网络、残差连接和归一化。残差让每一层在已有表示上修正，归一化帮助维持适合优化的数值尺度，前馈网络对每个位置做非线性变换。若使用混合专家（Mixture of Experts，MoE）

架构，路由器只激活部分专家；这能改变计算与参数容量的配比，但还带来通信、负载均衡和部署问题，并非免费扩大模型。

我们把常见的先归一化（pre-norm）解码器层画在图 2.5。第一条支路通过注意力从其他允许位置取信息，再加回原状态；第二条支路通过前馈网络变换各位置，再加回。图中的归一化位置是这一具体结构的约定，不应推广为所有 Transformer 都完全相同。

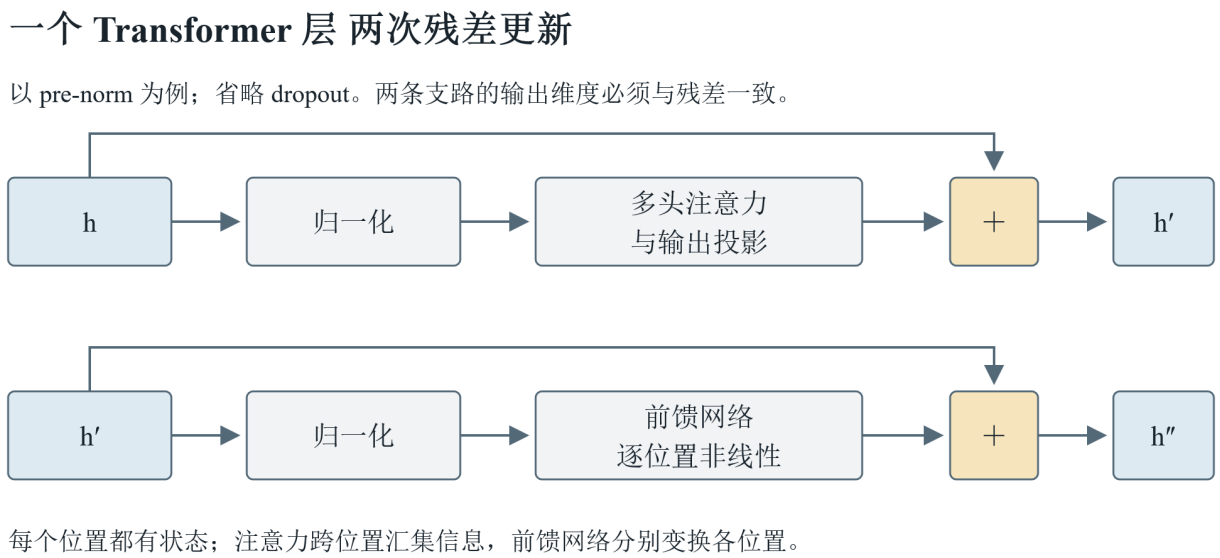


图 2.5 一个 pre-norm Transformer 层的两次残差更新。每次加号两侧形状必须一致；多层堆叠后才形成供输出头使用的上下文表示。

经过多层计算，当前位置的隐藏向量被语言模型输出头（Language Model Head，LM head）映射成词表大小的一组未归一化分数（logits）。logits 还不是概率；softmax 才把它们变成总和为 1 的分布。某些模型把输入 embedding 与输出权重绑定，某些不绑定。我们现在能把一轮生成完整地走通：token ID、输入向量、上下文表示、logits、概率，再到下一个 token。

配合视频理解若 Q、K、V 的矩阵关系仍然抽象，我们可以在 B 站搜索 “李宏毅 2021 自注意力机制 Self-attention”，选择包含完整计算过程的课程视频。边看边暂停，画出一次 query 与所有 key 比较、再加权 value 的过程。

本节相关资料

Vaswani 等，2017，Attention Is All You Need。原文链接 。**点积注意力里程碑论文，但现在所有的注意力实现都已经不再用这种形式，而是改用 GQA 等高效注意力变体，可自行学习**。阅读重点：注意力计算、多头结构与残差。原论文是编码器解码器结构，阅读时区分它与纯解码器语言模型。

Ba 等，2016，Layer Normalization。原文链接 。阅读重点：归一化解决哪一类优化困难。

苏剑林等，2021，RoFormer Enhanced Transformer with Rotary Position Embedding。原文链接 。阅读重点：RoPE 怎样通过旋转查询和键，把相对位置关系带进注意力分数。**中国最重要的大模型研究成果之一，苏剑林博客非常值得阅读**。

2.4　预训练与缩放定律怎样分配训练资源

我们终于可以训练一个 toy 模型。第一轮输出近乎乱码，训练一段时间后出现词语，再后来句子逐渐连贯。这个过程容易让我们只盯着损失（loss）曲线。可要理解曲线，我们需要先看一次参数更新。

我们从语料抽一个小批次（batch）[3]，前向得到预测概率，计算交叉熵，再反向传播得到每个参数对损失的梯度。优化器按学习率调整参数。梯度表示当前位置的局部变化方向，不是对整段知识的显式存储指令。学习率太大可能让训练不稳定，太小可能在有限预算里进展缓慢；有一些缓解方案可以了解：自适应矩估计（Adaptive Moment Estimation，Adam）优化器的解耦权重衰减版本 AdamW（Adam with Decoupled Weight Decay）还会维护优化状态，因此训练显存不只是权重占用。过去很多年工程实践中最常用的是 AdamW，近期出现了一个非常有前景的方法叫 Muon（本节参考论文 3），正在被工业界大规模应用，可以自行了解。

预训练的数据管道同样属于训练方法。我们先清洗编码和模板噪声，去除大量重复，再考虑来源质量、语言比例与许可要求。重复数据会改变训练权重；污染评测集会制造虚假的泛化。把更多低质量文本送进训练，既不等于更多有效知识，也不保证更低的目标任务错误率。

当计算预算有限时，我们有三个可调节预算的选择：模型多大、训练多少 token、使用多少计算。**缩放定律（scaling laws）**描述在一定实验范围内，损失随这些量变化的经验规律。论文 Chinchilla（本节参考文献 1）的重要启发是，**固定训练计算下，参数和数据需要共同增长**；只扩大模型却训练不足，会浪费潜力，比如对于一个 80B 模型的训练数据，用来给一个 800B 的模型训练，模型的最终表现就不会充分发挥。它给出的是在特定设置中拟合出的配比，不是适用于任意数据、架构和部署成本的永久常数。

我们可以把问题变成实验：用同一总计算预算，比较一个大模型少训练与一个小模型多训练。评价时既看验证损失，也看讲题任务。若大模型训练损失更低却验证更差，可能涉及过拟合、数据覆盖或比较预算不公平，不能仅凭参数量推断胜负。

部署会改变最优选择。如果助手要服务许多用户，稍小模型可能值得训练更久，以节省每次推理的成本；如果任务允许调用更强模型，路线又会不同。训练 scaling 与推理时增加采样或思考预算解决的是不同资源分配问题，我们在第七部分再把它们接起来。

本节相关资料

Hoffmann 等，2022，Training Compute-Optimal Large Language Models，也就是 Chinchilla。原文链接 。阅读重点：固定计算预算如何分配给参数与数据。

Loshchilov 与 Hutter，2017，Decoupled Weight Decay Regularization。原文链接 。阅读重点：AdamW 中权重衰减与梯度更新的关系。

Liu 等，2025，Muon is Scalable for LLM Training。https://arxiv.org/abs/2502.16982。

[3]训练时不会一条一条训，而是一次训多条样本，称为一个 batch，与传统机器学习概念一致。

2.5　幻觉与前缀错位为什么会连贯地写错

预训练后，小助已经能流畅解题。我们问一个编造出来的课程规则：“本课允许迟交几天？”，实际上不允许迟交，但是它也给出一个听上去合理的数字。我们没有把规则交给它，它依然给出答案。这让我们意识到：预测 token 和确认一个事实对大模型来说是不同的事情。

参数里可以存储和组织大量知识[4]，但生成时没有一个自动运行的“事实查表保证”。语料可能过时、相互矛盾或压根不含这个问题的答案。上下文又常暗示“这里应该继续回答”。因此，模型可以生成结构完整但依据不足的内容。我们不能把幻觉说成每个 NTP 模型都必然出现的严格数学定理；更有用的结论是，NTP 本身没有提供逐条事实保证。

另一个风险来自前缀。训练时，我们通常把真实序列的此前 token 交给模型，监督训练下一位置是否出现正确的 token。推理时，前缀则包含模型自己刚生成的内容，这就是自回归生成。

小助解题时若第一步误写成“21 除以 3 等于 6”，这一步已经错了，但是后续生成的内容依然看起来合理。训练数据里的标准解从未出现这个前缀，模型不一定学习过从这里纠正错误。训练前缀和实际生成前缀的错位，是暴露偏差（exposure bias）所指的一类问题。

我们把三种幻觉错误分开。1. 上下文缺少规则，主要是知识或信息获取问题，即上下文不充分；2. 知道题意却输出错误步骤，可能涉及模型学习过的计算与策略；3. 第一步错后持续自洽生成，涉及推理错误轨迹的自我恢复与训练分布。对应的实验也不同：1. 提供真实资料，观察能否纠正事实；2. 提供正确中间步骤，观察能否完成剩余推导；3. 故意放入错误前缀，观察能否发现并修正。这里的幻觉问题也是许多公司在面试时经常问的问题。

注意，大模型幻觉之前以及现在都是一个主流的研究方向之一，但随着模型本身智能的增强，幻觉问题很大程度上被缓解，这意味着一些热门问题可能是阶段性的，甚至在一定的历史条件下会被完全消除。

本节相关资料

Zhang 等, 2025, Detecting hallucination in large language models through deep internal representation analysis，https://arxiv.org/abs/2405.19648v1。我们组张鸾在 2025 年发表的关于幻觉检测的论文。

[4]大模型训练属于一种信息压缩，把远超参数量的信息压缩到有限的参数中

第三章　嵌入表示与语义检索

除了 3.2 以外可略读，3.2 讲解如何训练 BERT 模型，这种模型专注语义嵌入，在许多场景中仍然重要。还要理解 BERT 模型和 Transformer 模型的差别是什么，这种差异从何而来。

3.1　从关键词匹配到语义向量为什么文字不同也能找到答案

第二部分结束时，小助已经可以把一段问题接着写下去。我们现在给它增加一个很朴素的任务：学生问 “为什么不等式两边乘负数要变号”，请它先从课程资料里找出相关解释。资料明明写着 “一个不等式的两边同乘负数，不等号的方向必须反转”，简单的逐文字匹配却可能把这段排得很靠后，两句话字面上差距很大。问题不在于小助不会说话，而在于它还缺少一种寻找材料的办法。

最直接的方法是数一数词语出现了多少次。把查询和文章切成词，比较共享的词，词频高、在整个资料库中又较少见的词通常更有区分力。词频 - 逆文档频率（Term Frequency–Inverse Document Frequency，TF-IDF）和最佳匹配 25（Best Matching 25，BM25）都是这条路线上的代表。它们并非只能比较两个原始字符串：文本先被切分和赋权，然后才计算匹配分数。但如果学生换了说法，材料恰好用了另一种表述，关键词的重合程度就不一定能准确代表相关程度。

我们想让 “同乘负数要改变不等号方向” 和 “负数乘不等式为什么变号” 容易匹配，同时又让 “负数相乘的运算法则” 和 “不等式两边同乘正数” 与它区分开。这里要求的不是 “文字看上去像”，而是 “这段资料能不能解决这个问题”。两个句子可能谈同一个话题，却只有其中一个真正回答了问题。

一种办法是把文本映射成固定长度的数值向量，称为嵌入表示（embedding）。我们用 f 表示这个映射：输入一句话或一段资料，输出一个 d 维向量。d 是模型约定的维度，例如几百或上千维，并不是一句话的字数。每个维度的数值没有天然固定的语义，向量之间的关系来自模型的训练。

$$e \;=\; f_\theta(\text{text}) \;\in\; \mathbb{R}^{\mathrm{d}}$$

式中 text 是输入文本，e 是最后得到的向量，θ 是产生向量的模型参数。注意，向量化不是稠密嵌入的独有本领。独热表示（one-hot）同样用数字表示文本：如果词表有一万个词，就可以把每个词表示成一万个位置中只有一个位置为 1 的向量。它能准确标记词的身份，却无法仅凭这种表示把 “改变方向” 和 “反转不等号” 判断为语义相近。稠密嵌入把可学习的信息压进较少的维度，期待相关文本在某种打分规则下更容易靠近。

所谓 “靠近” 需要一个具体指标。例如余弦相似度（cosine similarity）比较两个非零向量的方向，而不是直接比较字面词语。对两个句子向量 u、v，有：

$$cos(u,\ v)\ =\ \frac{(u\ \cdot\ v)}{\|u\|_2\ \|v\|_2}$$

点号表示各维相乘后求和，双竖线表示欧氏长度。若先把向量归一化到单位长度，余弦相似度就等于它们的点积。不过并非每一种模型都用余弦，也并非所有任务都需要归一化。真正重要的是：训练时怎样规定相关性，使用时就要按相应方式比较。

我们做一个没有训练的思想实验。给三段材料随意安排三个二维坐标，即使画出来很漂亮，也没有理由相信正确答案离问题最近。几何本身不会产生语义；**是训练信号让某些语义关系逐渐反映到几何关系里**。因此，现在的问题不再是 “怎么计算两个向量的距离”，而是 “模型为什么会把应该匹配的文本放近”。

早期的 word2vec 通过词与上下文的共现规律学习词表示，说明仅靠语言数据就可能形成有用的几何结构。它通常为一个词学习较固定的表示；同一个英文 bank 出现在 “存钱的银行” 和 “河岸” 里，却有不同含义。上下文模型能够依据整句话改变某个位置的隐藏向量。我们在第二部分学过的 Transformer 已经具备这种上下文计算能力，但它的输入嵌入表、每个 token 的隐藏状态、表示整段文本的检索向量，仍然是三个不同对象，这告诉我们训练目标不同，嵌入表示的作用也就有差异。

3.2　Bert 模型怎样把相关文本训练得更近

我们已经知道，训练不是给每个词写一个固定的 “意思坐标”。那能否直接取出小助第二部分学到的 Transformer 层，把一句话送进去，再把隐藏向量平均一下，立刻得到好用的检索模型？我们试了试，发现有时同主题但不能回答问题的材料排在前面，真正有用的答案却排在后面。

这不奇怪。Transformer 这种自回归语言模型主要学习 “在此前 token 给定的情况下，下一个 token 应该有怎样的概率”，这就是 Transformer 模型的目标，换句话说，Transformer 模型不是为了语义分类这个目标训练的，而 Bert 模型就是为了这个目标训练的。会预测后续文字，不等于天然会给相似的句子之间做相关性排序。

要把已有模型变成用于检索的嵌入模型，我们至少需要两样东西：先定义**如何从一整段文字得到一个固定长度向量**，再定义**什么样的向量关系算训练成功**。

第一件事从架构开始。文本经过 tokenizer 变为 token ID，再经过 Transformer 产生每个位置的隐藏状态。我们可以取一个约定位置的状态，或对若干位置的状态求平均，把长度不一的句子变成同样维度的向量；这一步通常叫池化（pooling）。模型还可能再接投影层（projection），最后按训练约定进行归一化。对于使用 BERT 一类双向编码器的模型，可以利用双向上下文；对于以因果解码器为基础的模型，也可以通过特定位置、提示和训练方式得到句向量。没有一种池化方式适合所有架构。

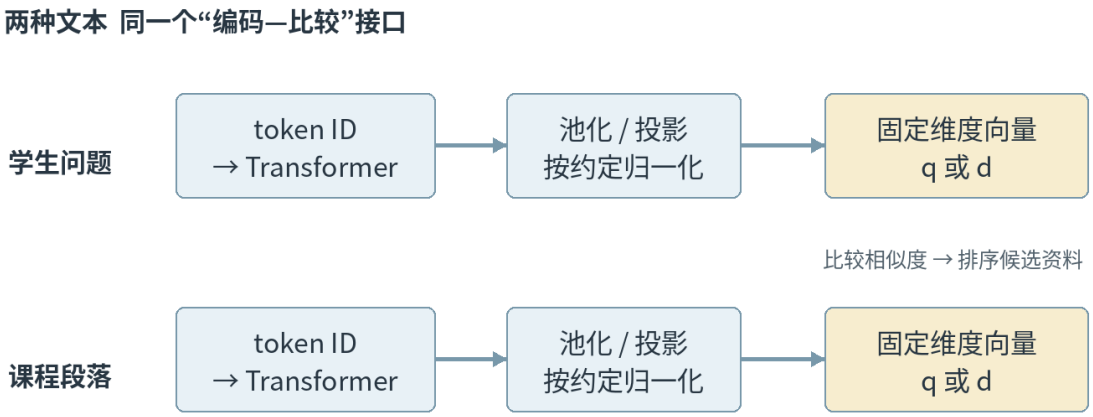


图 3.1 从整段文本到检索向量。输入 embedding 是 token ID 的查找结果；Transformer 的隐藏状态随上下文变化；池化以后才得到用于比较的整段文本向量。图示只是一种常见实现，具体模型还可能使用不同的提示、投影与归一化。

现在我们用同一个编码过程处理 “问题” 和 “资料”：一个编码器得到查询向量 q，另一个编码器得到文档向量 d，再比较二者的相似度。这种分别编码的结构常称为双编码器（dual encoder），或形象地叫双塔结构。两边可以共享参数，也可以使用不同参数；有些模型还要求给查询加 “query:” 前缀、给文档加 “passage:” 前缀。这些不是装饰文字，而是训练时输入接口的一部分。

为什么要分成两边编码？如果课程库有十万条材料，可以先把十万条资料都编码一次并保存。学生每问一道题，只需要编码新问题，再与已经保存的资料向量比较，这就是 RAG 的常用方案。另一条路线是把 “问题 + 候选段落” 一起送进模型评分，通常称为交叉编码器（cross-encoder）。后一种方式能够让问题和段落逐 token 交互，可能更精细，却难以对整个大型资料库逐条重算。双编码器牺牲了一些交互表达能力，换来了大规模预编码和快速检索。

第二件事才是训练目标。我们拿出一道查询 q：“为什么负数乘不等式两边需要变号？” 准备一段正确资料 d^+：“两边同乘负数会颠倒大小关系，故需反转不等号。” 再找两段不适合回答它的资料，例如 “等式两边同乘一个数仍相等” 和 “负数乘法的符号规则”，记作 d_1、d_2。模型每次分别编码它们，给出分数。我们希望正确段落的分数高于其他候选，这样就可以训练模型，使得模型的嵌入能力实现此目标。

如果我们只告诉模型 “这两个句子很像”，它可能学会把所有数学句子都放在一起。更有效的办法是让它在同一批候选里作出区别。这就是对比学习（contrastive learning）的直觉：不是预先规定每个向量的坐标，而是通过 “谁应该更近、谁应该更远” 的比较，反向更新编码器。

一种常见训练目标是信息噪声对比估计（Information Noise-Contrastive Estimation，InfoNCE）。为简化记号，假设一个查询对应 K 个候选文档，其中 d^+ 是唯一被标注的正样本，s(q,d) 表示相似度，τ 是正的温度参数。单个查询的损失可以用 softmax 函数写成：

$$L = -log\{exp\frac{s(q,d^+)}{\tau} / \sum_{\{j=1\}}^{K} \exp\frac{s(q,d_j)}{\tau}\}$$

分母把 K 个候选都算进去，包含正样本；分子只取正样本。这个比值可以看成：在当前候选集合里，模型给正确材料分配了多大的相对概率，前面加了个负号，所以通过梯度下降优化 L，就实现这个概率的提升。若四段资料分数都一样，模型只能给正确段约四分之一的概率；若正确段的分数明显高于其他段，损失就会下降。梯度从损失传回池化、投影和 Transformer 中实际参与训练的参数，使下一次编码后的几何关系有所改变。它不是把某条资料的坐标人工移动一下。

τ 控制分数差异在 softmax 中被放大的程度。较小的 τ 会使高分候选之间的差异更突出，但也可能放大错误标签或造成训练不稳定。实际训练还要决定 batch 大小、向量归一化、学习率，以及哪些参数更新。我们暂时不背配置，先看一条样本怎样提供有效信号。

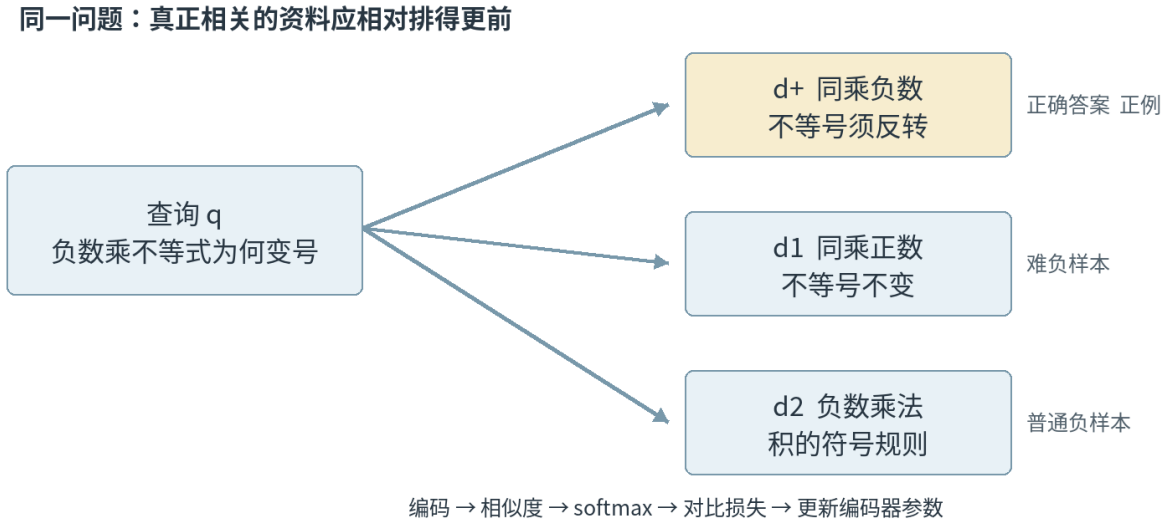


图 3.2 一次查询的对比学习。正样本要相对提高分数，负样本提供比较对象。与问题措辞接近却答错关键条件的难负样本，往往比完全无关的材料更能揭示模型是否学会了正确区分。

一个 batch 中还可以放入多道查询及各自的正确资料。对查询甲而言，其他查询的资料可以当作负样本，这叫批内负样本（in-batch negatives），让一次前向计算产生许多比较。但它有一个风险：某段“别人的正确资料”其实也能回答查询甲。我们误把它标成负样本，训练便会错误地把两段有效答案推远。这种情况称为假负样本（false negative）。因此，更多负样本并不自动意味着更高质量。

我们再让小助检索“负数乘不等式为什么变号”。一种难负样本（hard negative）是“等式两边同乘负数仍保持相等”：出现了很多相同的词，也包含正确数学叙述，却没有解释不等式的方向变化。模型若能把这样的材料排在真正答案后面，才说明它学到了超出关键词重合的区别。难负样本可以从 BM25 或旧版嵌入模型的高分误召回中挑出，但必须核查它确实不满足问题，不能把另一种正确解法误判为负例。

训练数据从哪里来？可以人工编写问题与答案段落；可以从人工确认的问答资料中整理；也可以让

强生成模型根据已有段落提出问题，再对生成结果进行核验。大量弱监督文本对可用于建立广泛的检索能力，之后再用目标课程里的高质量问题 - 资料对做适配。两阶段只是常见思路，不是所有嵌入模型必经的固定流程。点击日志也能提供信号，但点击往往受展示位置影响，不能直接当作无偏的相关性真值。

有了这些材料，我们才说清了“怎样训练 Embedding 模型”：选择预训练过的编码骨干，定义句向量的产生方式，组织查询、正例和负例，用对比目标计算损失并反向更新，再拿未见问题评价排序。如果直接从零训练也不是不可能，只是通常需要更大量数据与计算。一个检索模型是否好用，最终取决于它对目标查询的排序，而不是其向量图看起来多么整齐。

本节相关资料

Reimers 与 Gurevych，2019，Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks。原文链接。阅读重点：为什么需要专门面向句子相似度训练的模型，而不是把普通语言模型的表示直接当作检索结果。**这类做句子级嵌入的模型称为 SentenceTransformer，在现代自然语言任务中仍然常用，比如 https://huggingface.co/Xenova/all-MiniLM-L6-v2**。

3.3　从语义向量到真正的检索为什么不应直接淘汰关键词检索

我们训练出了一个嵌入模型。小助输入问题，模型给出查询向量；资料库里的段落也都有向量。接下来似乎只需挑出余弦相似度最高的几段材料。但在开始宣布“语义检索替代了旧检索”之前，我们给它增加两道题，看看它会在哪里失败。

第一道题是“怎么解释负数乘不等式要变号”。课程材料可能写作“同乘负数须翻转不等号”。两句话表面措辞不同，意义与任务却对应。训练合适的稠密检索（dense retrieval）往往能弥补词面匹配的不足，特别是在改写、同义表达和跨句联系比较多的场景。这里的收益不是因为向量计算比字符串计算更神奇，而是编码器在大量文本对上学会了什么样的相关关系。

第二道题是“请找出课程文件中的习题 A-017”。如果资料库恰好含有“习题 A-017”和“习题 A-018”，它们在主题与句式上可能极为接近；向量却未必把编号的单字符差异视为最重要的区别。此时基于关键词的 BM25 或精确字段匹配可能更加可靠。课程版本号、日期、代码标识符、人名和罕见术语也常有类似问题。使用 Embedding 并不意味着可以放弃精确匹配。

我们因此不把检索方法排列成“越新越好”的清单，而是按错误类型选择。词法检索依赖文本中的明确词项，适合精确名称和稀有关键词；稠密检索根据学得的向量关系，适合字面不同但任务相关的表述。两者可以并行取候选，再把结果融合，这叫混合检索（hybrid retrieval）。常见的做法是合并排名或按校准后的分数加权；不同系统的原始分数通常不能不经处理就直接相加。

在资料只有几百段时，最简单的做法是把查询向量与每个资料向量都算一次相似度，得到精确的向量近邻排序。如果资料变成百万段，这样逐条比较就会消耗较多时间和计算。传统机器学习方法中，近似最近邻（Approximate Nearest Neighbor，ANN）索引通过组织向量、减少需要比较的候选，换取更高检索速度，但可能漏掉真正的高分段落。因而 ANN 解决的是“怎样更快找到已有向量的近邻”，

不是“怎样训练出更有语义的向量”。

我们把整个过程按先后顺序写清楚。离线阶段，先整理资料，按适当段落切分，为每一段保存标题、来源、版本和标识，再用同一嵌入模型编码并建立索引；在线阶段，把学生问题按查询侧的约定编码，找出 top-k 候选，需要时合并关键词检索结果。若前面还有很多候选，也可以让问题和文档共同输入更精细的重排模型（reranker），再决定最后保留哪些资料。检索的输出是“值得阅读的候选”，并不是已经核实的最终答案。

这条链路会引出两类看起来相同、原因却不同的失败。假如一段正确材料在全量向量暴力搜索时就排在很后面，那么需要检查训练数据、查询写法、文本截断或编码模型；假如暴力搜索能找到它，但 ANN 查询遗漏了它，就优先检查索引设置和搜索预算。如果正确材料本来不在库中、版本已经过期，换一个更强的嵌入模型也找不回不存在的证据。

我们还需要防止“相似”被当作“正确”。学生问“作业最晚什么时候提交”，模型找出一段内容为“上学期作业截止时间”的文字，主题完全吻合，却不能回答当前日期。语义向量往往能指出讨论的是同一种事情，但未必可靠处理新旧版本、权限或细粒度条件。建立资料库时保存并使用版本、时间和来源，必要时做明确过滤，比盲目提高相似度阈值更有用。

现在给小助设计一次可以复核的检索实验。我们先标注二十个尚未参与训练的课程问题，以及每题真正能够回答它的段落。让 BM25、嵌入检索和混合检索使用同一份资料，分别记录前五名有没有出现正确证据。如果我们只为每题选定一段标准证据，可以报告 Hit@5（前五名命中率）：看有多少题在前五名里找到了指定的正确段落。若给每题标注全部相关段落，也可以报告 Recall@5（前五名召回率），这时应计算找回了多少条相关段落、共有多少条相关段落。不要混用指标口径。

如果 Embedding 在改写问法上优于 BM25，却在编号题上退步，这不是“实验失败”，而是找到了它的适用条件。我们再查看失败样本：正确文档是否存在，相关性标注是否准确，向量模型是否把同主题错误答案排得过高，ANN 是否漏召回，混合检索是否真的互补。三种方法要在相同候选库、相近延迟或明确的计算预算下比较，不能让其中一种悄悄多看资料。

同一个嵌入空间还可以辅助找近似重复资料、组织主题或筛选异常样本。但“两个段落很像”不足以说明其中一个该被删除，“二维降维图上分成三团”也不足以证明主题一定正确。每换一种使用方式，我们都要重新定义要保留什么信息、要接受什么错误。

走完这一轮，小助终于拥有了在资料库里找线索的能力。我们理解了它来自怎样的训练：预训练模型提供上下文表示的基础，池化等操作给整段文字一个可比较的向量，文本对与对比学习让“能回答问题”逐渐变成可排序的关系，检索系统再把这种关系用于实际资料。与此同时，找到材料不等于已经读懂材料，更不等于自然会按学生的要求讲解。对于已经找回的证据，小助还需要学会怎样作答。

3.4 基于大语言模型的现代 Embedding

我们已经让小助能够从课程资料中寻找证据，但还有一个问题没有展开：第二章训练的是善于预测下一个 token 的大语言模型，为什么到了语义检索，却好像必须换成 BERT？如果我们手里已经有一个训练充分的大模型，能不能利用它读过的大量文本，直接训练出更好的检索向量？现代 Embedding

的一条重要路线，正是把预训练的大语言模型改造成文本编码器。本节沿着一次改造，看看向量从哪里来、相关性怎样学会，以及这种做法能解决什么问题。

先把架构与训练目标分开。BERT 和我们在第二章讨论的纯解码器（decoder-only）语言模型都建立在 Transformer 上。BERT 通常使用双向注意力，并以掩码语言建模等目标预训练；因果语言模型则遮住未来位置，主要学习下一个 token 的条件分布。BERT 的预训练也不是直接学习 “哪些句子应该更相似”。两类骨干都可以经过专门训练得到句向量。因此，现代 Embedding 的变化，是利用另一种预训练起点，再重新规定输入、读出方式和优化目标。

从预测下一个 token 到读出整段文本

我们先把问题 “为什么不等式两边乘负数要变号” 送入一个因果语言模型。tokenizer 得到长度为 T 的序列，输入嵌入表把各个 ID 变成向量，多层 Transformer 再输出 T 个上下文隐藏状态。设最后一层输出为：

$$H = (h_1, h_2, \dots, h_T), \qquad h_t \in \mathbb{R}^m.$$

这里的输入向量是查表结果，h_t 是结合上下文计算出来的状态；我们最终用于检索的整段文本向量，还需要从这些状态中读出。

生成时，我们把隐藏状态交给语言模型输出头，得到词表上的 logits，再预测后续 token。做 Embedding 时，常见做法是在输出头之前取隐藏状态，经过池化，必要时接投影，再按模型约定归一化。模型输出的是数值向量，通常不需要像聊天一样逐 token 生成一段回答。

因果掩码使第一个位置只能读到开头，后面的状态可以读到更长前缀，最后一个有效位置则能够关注整段已输入文本。这给出一个自然的读出办法：取最后一个有效 token 的最终隐藏状态，称为末词元池化（last-token pooling）。若最后一个有效位置为 t_{last}，则：

$$z = h_{t_{last}}.$$

如果训练约定在正文后追加序列结束标记（End of Sequence，EOS），我们就取 EOS 所在位置的状态，这通常叫 EOS pooling：

$$z = h_{EOS}.$$

EOS pooling 是一种特定的末位置读出，二者不能在所有实现中直接画等号。[3][4]

这里的关键是 “这个位置能读取全文”，而不是 “最后一个词本身概括了全文”。假设所有问题末尾都加同一个 EOS，查表得到的 EOS 输入向量确实相同；但经过多层注意力后，它在不同问题中的隐藏状态可以不同，因为它汇集了不同的前文信息。后续的检索训练还会进一步推动模型，把任务需要的信息组织到这个读出位置上。

我们也不能反过来说，只要能看见全文，就已经把全文无损保存下来。一个固定长度向量需要压缩

信息，训练会决定哪些差异值得保留。最后位置在因果结构上具备汇集全文信息的条件，并不保证未经适配的模型已经学会把 “能回答问题” 表现为余弦相似度。直接提取普通聊天模型的末位置状态可以作为基线，却不能据此认定已经得到高质量检索模型。

实际编码还有一个容易忽略的细节。不同长度的文本放进同一个 batch，通常要用填充 token（padding token）补齐。如果我们机械地取数组最后一列，就可能取到填充位置。应当依据 attention mask 和填充方向，找到最后一个有效位置；若模型要求 EOS，还要确保截断后保留了约定的结束标记。结束符、模板、截断和池化必须一起遵守模型接口，不能只看 “输出维度相同” 就认为实现等价。[4]

末位置读出也不是唯一选择。平均池化（mean pooling）可以汇总多个位置。设参与池化的有效位置集合为 $\mathcal{V}$，则：

$$z = \frac{1}{|\mathcal{V}|} \sum_{t \in \mathcal{V}} h_t.$$

这里的有效位置通常不包含 padding，是否包含指令和特殊标记则由模型的训练约定决定。某些因果模型也会使用位置加权平均，让看到更长前缀的位置得到更大权重。另一条路线则改变注意力掩码，使每个位置都能读取左右上下文，再进行平均池化。哪一种更合适，要看模型怎样训练，不能推导出 “LLM 一律用 EOS” 或 “改成双向就一定更强”。

把大模型放进双塔以后怎样训练

现在我们把问题和资料分别送入这个编码过程。查询侧得到 e_q，资料侧得到 e_d，再通过点积或余弦比较。这仍然是前文的双编码器，也常叫双塔结构。“双塔” 描述的是两段文本分别编码，不要求存在两份参数不同的大模型。查询和资料可以共享同一套权重，只使用不同的输入模板；由于查询侧含有任务指令，即使共享参数，两侧的实际输入与功能也可以不同。

我们可以用一组简写串起整条计算链：

$$H_q = LLM_\theta\left(template_q(I, q)\right), H_d = LLM_\theta\left(template_d(d)\right),$$

$$z_q = pool\left(H_q\right), z_d = pool\left(H_d\right),$$

$$e_q = \frac{z_q}{\left\|z_q\right\|_2}, \quad e_d = \frac{z_d}{\left\|z_d\right\|_2},$$

$$s(q, d) = e_q^\top e_d.$$

I 表示任务指令，q 是查询，d 是资料，θ 是共享的骨干参数。template 表示输入格式约定，pool 表示池化。这里选用归一化后的点积作为示例，暂时省略可选投影层；并非每个 Embedding 模型都采用完全相同的接口。真正需要记住的是，查询和资料独立得到向量，评分时才把二者联系起来。

假如小助要查“为什么不等式乘负数要变号”，我们准备正确段落 d^+，再准备一个措辞很像却只讨论“等式两边同乘负数”的 d^-。原来的语言建模目标主要要求模型提高真实下一个 token 的概率；现在的目标则要求查询向量与正确段落的相似度，相对高于错误段落。这是从生成训练转向表示训练的关键一步，换掉池化操作本身并不能完成这种转变。

仍以前文的 InfoNCE 为例，设候选集合 $\mathcal{C}$ 包含正例和若干负例，单个查询的损失可写为：

$$\mathcal{L} = -\log \frac{\exp\left(s\left(q, d^+\right)/\tau\right)}{\sum_{d_j \in \mathcal{C}} \exp\left(s\left(q, d_j\right)/\tau\right)}.$$

τ 是温度参数，分母包含正例。如果正确段落和错误段落得分相同，模型就没有学会区分它们；降低损失会推动正确段落获得更高的相对分数。

梯度经过相似度、归一化和池化，回到参与训练的 Transformer 参数。我们可以全量更新骨干，也可以使用低秩适配（Low-Rank Adaptation，LoRA）等方法，只更新一部分附加参数。具体哪些参数改变由训练配置决定，并不是只训练输入嵌入表。

向量坐标因此随着编码函数一起变化。某次参数更新后，许多文本重新编码的结果都会改变，而不是只移动这次训练样本的坐标。部署时，资料库可以离线预编码，在线只编码新问题；但换了模型检查点、池化或资料模板，就应重新生成兼容的资料向量和索引。两个模型即使输出同样维度，也不意味着它们共享同一个坐标空间。

数据决定“相关”究竟是什么意思。同一句“负数运算”既可以按主题分类，也可以用于寻找解题证据。我们给查询增加“查找能够解释该数学问题的课程段落”这样的任务指令，就是要告诉模型当前应保留哪一种关系。指令感知的 Embedding 通过带任务说明的文本对训练，让指令能够影响最终向量。[3]

这类训练样本可以组织为：

$$(I, q, d^+, d_1^-, \ldots, d_K^-).$$

正例不必是查询的同义改写，也可以是一段真正回答问题的资料。负例可以来自批内其他文本，或者来自旧检索器召回的高分错误材料。我们仍要检查假负例：如果另一个段落也能解释变号，却被当成负例，模型就会收到相互冲突的信号。困难与正确是两件事，不能只按分数高低自动决定标签。

生成式大模型还可以参与构造这些样本。我们让它根据真实课程段落提出不同问法，覆盖初学者、不同语言和不同难度，再筛除答非所问、遗漏条件或与原文矛盾的配对。这时大模型可能承担两种角色：一个作为待训练的编码骨干，另一个作为数据生成器或筛选器。两种角色可以分开，不需要让在线检索器同时运行一套生成流程。合成数据扩大了训练覆盖，但教师模型的错误和表达偏好也可能被带入，因此仍要用独立的真实查询检查泛化。

三条现代路线怎样作出不同选择

LLM2Vec 回答的是“怎样把已有因果语言模型改造成双向文本编码器”。它先开放双向注意力，再用

掩码下一词元预测（Masked Next Token Prediction，MNTP）适配新的信息可见方式，之后通过无监督对比学习改善序列表示，并使用平均池化。MNTP 保留了下一位置预测的移位形式，却针对被遮蔽的 token 计算训练信号；它不是简单把普通 BERT 的损失原样搬过来。需要时还可以进一步做有监督对比训练。[1]

我们可以把这条路线理解为：保留预训练权重积累的语言基础，调整信息流动方式，再让模型适应新的表示任务。它说明 decoder-only 是一种预训练起点，不是改造后永远不能改变的限制。不过，只在推理时随手删除因果掩码，并不能等价复现 LLM2Vec 的训练过程；新的注意力模式需要与相应训练一起理解。

GritLM 关注另一件事：能否让同一套模型权重兼顾生成与 Embedding？它通过生成表示指令调优（Generative Representational Instruction Tuning，GRIT），把表示任务的对比损失与生成任务的语言建模损失结合起来。我们可以把联合目标简写为：

$$\mathcal{L}_{joint} = \lambda_{rep}\mathcal{L}_{rep} + \lambda_{gen}\mathcal{L}_{gen}.$$

其中，$\mathcal{L}_{rep}$ 是表示学习损失，$\mathcal{L}_{gen}$ 是生成任务的语言建模损失，两个 λ 控制各自权重。这只是帮助理解的简写，实际训练还涉及两类数据的采样比例、批次组织与损失归一化。

做 Embedding 时，GritLM 采用双向注意力和平均池化；生成时保留因果注意力。池化只汇总正文对应的最终隐藏状态，指令和格式 token 不直接参与平均，但仍能通过注意力影响正文状态。[2]

因此，“统一模型” 不意味着同一次前向计算同时产生最佳向量和完整回答。两类任务仍有不同的输入、掩码与损失约定。GritLM 展示了共同训练的可行性，却不能证明任何聊天模型加一个池化层，就能自动兼顾两种能力。应用中是否值得共享模型，还要考虑服务调度、显存、延迟，以及我们是否真的同时需要两种功能。

Qwen3-Embedding 则展示了保留因果注意力的路线。其技术报告使用输入末尾 EOS 位置的最终隐藏状态作为文本表示，并通过多阶段对比训练适配相关性任务。查询侧包含任务指令，检索文档侧通常不加同样指令。它利用大规模合成文本对做弱监督训练，再结合高质量数据微调，并通过检查点合并改善泛化；不同阶段的对比数据质量和覆盖不同，不能把第一阶段简单理解为重新训练一次普通聊天模型。[3][4]

这条路线让我们看到，双向化不是 LLM Embedding 的必经步骤。Qwen3-Embedding 把任务需要的信息训练到因果模型的末位置读出上；LLM2Vec 则改变可见范围后汇总各位置；GritLM 进一步兼顾生成目标。三者共同利用大语言模型的预训练基础，但对注意力、池化和任务组合做出了不同选择。复现时必须采用各自的完整配方，不能把一个模型的权重与另一个模型的模板或池化随意拼接。

什么时候值得替换原来的编码器

现在我们可以回答开头的问题。大语言模型经过充分预训练，已经形成丰富的上下文表示，也可能具备较好的多语言和复杂语句处理能力；表示训练有机会把这些基础转化为更好的相关性排序。遇到跨语言问法、复杂条件或代码检索时，这种起点可能带来收益。但收益来自骨干、数据、任务指

令与优化的共同作用，不能只归因于“参数更多”，也不能从生成能力直接推导检索成绩。

代价同样具体。较大的骨干通常需要更多显存和编码时间，资料更新时还要支付重新编码成本。长上下文支持意味着模型能够接收更长输入，却不保证一个固定长度向量能保留长文所有细节；把整本教材压成一个向量，往往不如恰当切分后分别检索。嵌入维度还影响向量库容量和相似度计算成本，不能只比较模型参数量。

小型双向编码器因此仍有价值。若课程资料范围稳定、问法简单、延迟预算紧，小模型可能已经达到所需召回率。课程编号、公式符号和版本条件仍可能需要关键词或字段过滤；候选之间细微的条件差异，也可能需要交叉编码重排。LLM Embedding 扩展了可用的编码器路线，并不自动取代词法检索、小模型或重排系统。

我们可以给小助安排一个小实验：固定同一份课程资料和段落切分，比较普通因果语言模型的直接读出、已经训练好的小型嵌入模型，以及专门训练的 LLM Embedding。每个模型遵守自己的官方模板和池化要求；测试问题按改写、复杂条件、编号和跨语言分别记录。除前五名命中率外，还记录查询延迟、资料编码耗时和存储开销。若要判断“提升是否来自对比训练”，则需要在同一骨干和相同读出设置下比较训练前后，而不能仅比较两个名字不同的模型。

我们还应抽查那些主题相似却证据错误的结果。相关性分数不是事实正确率，也不是统一校准的置信度，不同模型的相似度阈值通常不能直接互换。如果大模型只在通用榜单上占优，在真实课程问题上却没有改善，我们就没有充分理由承担替换成本。相反，如果它能在可接受预算内稳定找回原先遗漏的证据，那么这才是更换编码器的实际依据。

走到这里，第二章和第三章之间的连接就清楚了：语言模型预训练提供可迁移的上下文计算能力，池化规定从哪里读出整段文本，指令与对比数据规定保留怎样的相关性，再由检索系统把向量用于寻找证据。小助可以继续使用独立的生成模型来阅读这些证据，也可以在经过专门训练的统一模型中切换任务。找到证据以后，怎样依据证据给出合适的回答，将继续把我们带向后面的训练与应用。

实践检索增强生成现在（RAG）已经成为一种基本技能，尝试完整总结 RAG 的实现流程。

本节相关资料

[1] BehnamGhader 等，2024，LLM2Vec: Large Language Models Are Secretly Powerful Text Encoders。阅读重点：双向注意力、MNTP 与对比学习分别解决什么问题。配合阅读官方教程。

[2] Muennighoff 等，2024，Generative Representational Instruction Tuning。阅读重点：生成与表示怎样共用权重，以及两类任务的掩码、池化与损失怎样区分。官方实现。

[3] Li 等，2025，Qwen3 Embedding: Advancing Text Embedding and Reranking Through Foundation Models。阅读重点：因果注意力下的 EOS 读出、指令输入与多阶段对比训练。

[4] Qwen 团队，Qwen3-Embedding-0.6B 官方模型卡。阅读重点：查询与文档的输入差异、last-token pooling、padding 与归一化。使用其他检查点时，应同时核对其分词器和官方配置。

第四章　监督微调与续训退化

4.1　监督微调 SFT 用示范塑造助手行为

回到最初的方程题，我们给出一条完整示范：“先把两边除以 3，得到 a + 2 = 7；再把两边减去 2，得到 a = 5；代回检验成立。”我们不只希望模型背下来这个答案，而是希望它在新题上也遵循解释、求解、检查的任务结构。

SFT 让模型在指令与对话上下文下，提高示范回答的概率。从目标看，它仍然通常是逐 token 最大似然；从数据看，它把我们关心的助手行为集中呈现给模型。示范可以由人写，也可以来自强模型或数据合成管线。“监督”指存在目标输出，不等于所有文本都必须由人类逐字编写。

我们用掩码（mask）标出哪些位置计算损失。若目标是教助手回答，常见做法是只对助手回复计分，把系统和用户内容作为条件；多轮训练也可以监督多个助手轮次。没有统一规定说所有 SFT 都必须只算最后一轮，关键是实际监督范围是否符合任务。把位置 t 的监督掩码记为 m 的下标 t，只对参与监督的 token 取平均，SFT 损失写为：

$$L_{SFT} = -\frac{1}{\sum_t m_t} \sum_t m_t \log p_\theta \left(x_t \mid x_{<t} \right)$$

求和号按序列位置相加，分母是有效监督位置数；若分母为 0，应拒绝该样本并检查数据处理，不能继续计算这个平均损失。m 为 1 的位置参与监督，为 0 的位置不参与，因为被 mask 掉的 token 的 log p 乘以 0 就直接是 0。

图 4.1 把两个经常混淆的 mask 分开。损失掩码决定哪些目标 token 直接计分，因果掩码决定预测这些目标时能读取哪些输入位置。常见实现会将输入与目标错开一位；若移位做错，模型可能在输入中直接看到被考查的答案。



图 4.1 一条 SFT 样本的目标与损失位置。用户 token 即使不直接计损失，仍能作为助手回答的条件，其相关参数也可能经后续回答损失收到梯度。真实分词和模板可能拆成更多 token。

从行为克隆（behavior cloning）的视角，SFT 告诉模型 “遇到这里，示范者这样走”。示范由外部给定，在本书的宽泛用法中属于异策略（off-policy）数据；它与学生当前生成的路径可能不同。它没有显式要求模型在多个候选中评判哪个更好。可我们也不能说它绝对学不到判断：若数据里包含批改、比较和自我纠错示范，模型同样可以学习这些行为。真正的区别是，普通最大似然并未直接优化独立的偏好比较或环境回报。

训练后，小助开始稳定给出分步答案。我们很高兴，却先在验证集核对：是否只学会了固定套话？是否每题都添加无意义检查？是否在资料不足时照样编造？格式是容易观察的进步，任务能力与可靠性还需要分别检查。

本节相关资料

Ouyang 等，2022，InstructGPT。原文链接 。阅读重点：监督示范在整个反馈训练管线中承担什么作用。

Wei 等，2021，Finetuned Language Models Are Zero-Shot Learners，FLAN。原文链接 。阅读重点：任务多样性如何帮助指令泛化。

4.2　SFT 数据工程模板与监督位置

我们准备扩充 SFT 数据。最容易的做法是搜集尽可能多的问答，却发现质量逐渐失控：有的答案只给数字，有的先闲聊，有的把推理写错，有的缺少轮次结束标记。模型会同时拟合这些差异，我们不能把它们都归结为 “数据越多越好”。

我们先规定示范质量。方程题要求步骤正确、足够解释且不编造定理；资料题要求引用可找到、依据能支撑回答；信息不足题要求说明缺少什么。我们保留不同的表达方式，避免把多样性误当成噪

声，但不保留彼此矛盾的任务要求。数据清洗因此不仅是去重，还包括明确我们希望稳定出现的行为。

训练前，我们随机抽取已 token 化的样本，检查角色边界、监督位置、最大长度与结束标记。若长样本总在最终答案之前被截断，小助就可能学会漫长推演，却学不到收束推导。

我们也会看分项损失。固定模板词频很高，模型很快学会“下面分步骤说明”，总 loss 因此下降，但真正关键的算术 token 改善有限。这不表示格式一定“劫持”了学习，而是平均指标掩盖了不同位置的难度与价值。我们把格式正确率、答案正确率、步骤有效率并列，防止只优化最容易的一项。

计算资源有限时，我们可以使用低秩适配（Low-Rank Adaptation，LoRA）。它冻结大部分基础权重，用低秩增量表达参数更新，减少可训练参数和优化状态。LoRA 解决的是适配成本与参数化方式，不会自动修复坏数据，也不保证没有遗忘。秩（rank）、作用模块、学习率和数据量仍需要验证。全参数微调也并非总优于 LoRA，比较应在清楚的预算和任务下进行。

第一次小规模训练，我们故意让模型在少量干净样本上过拟合。若连这一步都做不到，先查管线；若可以学会，再扩大数据，观察泛化。这是一个前期测试，不是最终评估。任何系统性工程任务都需要预实验，对模型的上限和下限都有所了解。

本节相关资料

Zhou 等，2023，LIMA Less Is More for Alignment。原文链接 。**阅读重点：高质量少量示范数据的训练价值可能非常大；但是也不要认为“少量数据总是足够”**。

Hu 等，2021，LoRA Low-Rank Adaptation of Large Language Models。原文链接 。（务必理解 LoRA 原理）**阅读重点：低秩更新节省了哪些资源，又没有保证哪些能力。即便不看论文原文也建议找资料理解 LoRA 如何高效训练**。

4.3　续训退化新任务变好为何旧能力变差

小助已经会耐心讲题，我们想加入一门新课程，于是准备一批领域问答继续 SFT。为了节省标注，答案几乎都是一句话：“答案是 5。”“定义见第三章。”“选择 B。”新课程题的分数上升了，但小助对旧题也开始只给结论，不愿解释。

小助可能仍然有能力解释这些问题，只是在新训练分布下更偏向简短回答；也可能失去了某些步骤能力，这就是“灾难性遗忘”[1]；也可能是对话模板变了（一般不太可能出现这种失误）。

更新为什么会波及旧任务？不同任务共享大量参数。新数据的梯度鼓励短答案，可能降低长解释的概率；新领域的词汇、格式、难度与采样比例，也会重新分配训练权重。这些改变并不局限在新题的专属区域。

训练强度同样关键。相同的新数据，较高学习率、更多轮次、较窄覆盖，可能造成更大的参数漂移。

[1]灾难性遗忘是指这种现象：一个经过后训练的大模型在各方面能力达到均衡，此时用一个专项领域数据继续 SFT，导致其他各方面能力明显下降。这是许多任务里面需要考虑的问题。

反过来，过低学习率也可能学不到新增任务。我们不能只说“把学习率调小”，而要在新任务收益和旧任务保留之间做曲线比较，用实验结果来验证猜想。

我们把旧评估拆成两种：正常提问，以及明确要求“逐步解释并检验”的提问。若后者大幅恢复，可能主要是回答行为变化；我们还可以给模型提供正确的推理的开头，如果模型仍无法完成任务，那么有可能新的训练已经损害了模型本身的能力。

MiniMax 的稀疏 token 案例为我们补充了一个具体假设：后训练前后，输入嵌入变化较小，而某些低频 token 的 LM head 向量发生偏移；它们仍可能被输入侧利用，却更难作为输出生成。研究团队报告，加入词表覆盖数据能改善其测试案例。

这时我们也重新考虑方法的选择。若希望模型掌握经常变化的课程规则，采用 RAG 方法也许比把每条规则写（训练）进权重更合适；若要长期改变模型的语言风格，SFT 更直接。继续训练之前，我们必须明确目标：希望模型新增知识还是改变行为，或者想都实现。

本节相关资料

Kalajdzievski，2024，Scaling Laws for Forgetting When Fine-Tuning Large Language Models。原文链接 。阅读重点：微调收益与原能力保留如何随训练条件变化。

Wu 等，2023，From Language Modeling to Instruction Following Understanding the Behavior Shift in LLMs after Instruction Tuning。原文链接 。阅读重点：指令微调前后的行为变化应如何测量。

MiniMax Research，2026，稀疏 Token 遗忘排查记录，研究博客。原文链接 。阅读重点：对照预训练与 SFT 的输入嵌入、输出头，并用覆盖数据验证修复方向。

4.4　遗忘诊断用对照实验选择修复方法

我们决定回到续训前的同一个检查点[2]，做几组可比较的实验。A 组只用新课程数据；B 组加入一部分旧任务示范；C 组保持 A 的数据而降低训练强度；D 组改善新课程示范，让它们包含必要讲解。每组只改变明确的一项，先避免同时换模型、模板、数据和学习率。在实际工作里面要定位具体的问题通常就需要这样做许多组实验来观察。

比较混合数据时，一个细节很容易破坏结论。若 B 组训练 token 更多，它的优势可能来自额外计算；若固定总 token，新领域数据暴露量又会减少。我们要说清比较回答的是哪一个问题：相同总计算下如何分配数据，还是相同新数据暴露下保留旧能力的成本是多少。最好分别报告，而不是把两个问题混成一个排名。

我们记录一组教学用结果：A 新领域提高最多、旧讲解下降明显；B 新领域略有改善、旧能力保留较好；C 退化较少，但新增知识也没学牢；D 同时改善讲解和新题。它们不证明混合数据或低学习率普遍有效，却告诉我们，这次数据风格与更新强度都可能参与了退化。

接下来我们检查失败样本。如果旧题的最终答案正确、解释变短，补充讲解示范可能有针对性；如果

[2]检查点一般是指之前的模型权重，每一个训练阶段都会保留一些检查点方便回溯，因为最新的训练可能出现问题

新题使用旧规则，可能需要更明确的领域条件；如果所有长回答都提前停止，则先查结束标记。修复手段必须与已观察到的缺陷相对应。

我们可以加入回放数据，保存代表性旧任务；也可以在旧输入上约束模型与参考模型的分布距离，或使用老师给出反馈。约束只能在我们检查的输入上发挥作用，不能保证保留全部能力。新旧目标若本身冲突，也不存在无成本的完美保留，我们需要依据实际使用需求确定取舍。

这一轮工作形成了后面反复使用的节奏：发现退化，分解指标，提出假设，设计单变量对照，检查样本，再扩大实验。我们不再把模型训练当作不停尝试超参，而是把它当作一个可以逐步排除解释的研究过程。

实践参考一些开源学习项目，在服务器上用 LoRA 方法 SFT 后训练一个 Qwen 小模型，观察训练前后模型的某些能力是否像预期一样变化。

本节相关资料

Kalajdzievski，2024，Scaling Laws for Forgetting When Fine-Tuning Large Language Models。原文链接。阅读重点：如何把遗忘问题转化为可比较的实验。

【人工校阅进度标记】

第五章　采样与强化学习

5.1　解码与采样多次尝试能发现什么

我们修好 SFT 后，挑了一批稍难的题。小助第一次回答错了，换个随机种子再答，却给出有效解法。我们没有增加知识，只改变采样；这说明一次输出不能完整描述模型在该题上的行为分布。

生成时，贪心解码（greedy decoding）每步选择最高概率 token，路径确定但未必最优。随机采样（sampling）按概率抽取，可能访问不同思路。温度通过缩放 logits 改变分布：温度较低通常更集中，较高通常更分散，但更分散不等于更多有效探索。核采样（nucleus sampling，也称 top-p sampling）只在累积概率达到阈值的候选中重新归一化采样，减少极低概率 token 的干扰。设前缀为 h、候选 token i 的 logit 为 z 的下标 i、正温度为 T，温度采样分布写为：

$$p_T(i \mid h) = \frac{\exp\left(z_i/T\right)}{\sum_j \exp\left(z_j/T\right)} \qquad T > 0$$

这里 j 遍历词表候选，分母保证概率总和为 1。此处的 T 表示温度，与第 2.2 节表示序列长度的 T 不同；top-p 截断是在得到分布后进行的另一道处理。

我们用 pass@1 描述一次尝试的成功情况，用 pass@k 描述 k 次尝试至少有一次成功的情况。若每次独立且单次成功率相同为 p，那么至少一次成功的概率是 1 减去 k 次全部失败的概率。真实模型不同题的成功率不同，输出也可能高度重复，所以总体上不能随便把平均 p 代入当成准确评估。在上述独立且等成功率的简化条件下，k 次中至少成功一次的概率写为：

$$P(at\ least\ one\ success) = 1 - (1-p)^k$$

其中 p 是单次成功概率，k 是尝试次数；右边先计算全部失败的概率，再用 1 减去它。这是简化模型，不是对所有实际采样分布都成立的通用估计器。

最关键的区别是：pass@k 假设我们有办法识别哪次成功。它不等于系统最终只返回一个答案的正确率。数学题可以用答案验证器，代码可以用测试；开放式讲解往往还要人工或可靠评审。没有选择器，多生成十份回答可能只增加成本。

我们做一个题目难度画像。某些题几乎每次都对，某些题偶尔对，某些题在预算内从未成功。中间那类题常适合从尝试中筛选或获得差异反馈；全错的题需要进一步检查能力、格式和奖励；全对的

题则可能提供较少的相对学习信号。有限采样没成功，只能说明“在本次预算和采样配置下未见成功”，不能证明成功概率严格为零。

我们现在有了一个连接训练与推理的观察：模型偶尔能走通的路径，可以变成后续训练的数据；而不同采样配置会改变哪些路径能被发现。接下来，我们先从最简单的筛选开始。

停下来试一试对二十道题各采样八次，记录正确次数和去重后的答案数。在相同 token 预算下比较两种温度。解释为什么更高 pass@8 未必意味着 greedy 更好，也未必意味着可部署的最终准确率更高。

本节相关资料

• Chen 等，2021，Evaluating Large Language Models Trained on Code。原文链接 。阅读重点：pass@k 的定义及基于有限样本的估计方式。

• Wang 等，2022，Self-Consistency Improves Chain of Thought Reasoning in Language Models。原文链接 。阅读重点：采样与答案选择如何共同影响最终效果。

5.2　拒绝采样微调 RFT 把成功尝试变成示范

小助已经在部分难题上偶尔成功。我们让它对每题生成多个解答，再用验证器筛出正确结果，用这些结果继续做 SFT。这样，原来稀少的成功路径在下一轮更容易被生成。这就是本书所说的拒绝采样微调（Rejection Sampling Fine-Tuning，RFT）。

这个缩写在不同语境也可能表示 reinforcement fine-tuning。我们阅读论文时必须展开全称，避免把筛选后最大似然与强化学习混成同一种算法。此处的更新损失仍然是最大似然，改变的是训练数据的来源与筛选规则。

我们先用当前检查点采样，筛选后更新，再用新检查点重新生成。与当前策略分布一致的采样称为同策略（on-policy）采样；筛选和后续参数更新又会改变这种对应关系。数据刚产生时接近当前策略分布；经过筛选，它已经条件化到成功子集；随着参数更新，旧样本也逐渐变旧。因此，我们称它具有迭代的策略相关性，比简单贴上“完全 on-policy”更准确。若只在最初生成一次，以后反复训练，它就越来越像静态数据微调。

验证器是这条管线的关键。只检查最终答案，可能留下碰巧答对却推导错误的样本。代码通过公开测试，也可能不满足未覆盖的边界条件。我们不仅要筛对错，还要检查答案解析、等价形式、测试覆盖和数据泄漏。对于开放任务，也可以用奖励模型或规则筛选，所以拒绝采样并非只适用于有唯一标准答案的任务；只是可靠筛选在可验证任务上通常更容易。

自学推理（Self-Taught Reasoner，STaR）让我们看到另一种办法：若模型直接解不出，可以在提供正确答案的条件下生成解释，再筛选有用的推理数据。这扩大了可获得示范的范围，也带来风险：知道答案以后写出的解释可能是事后合理化。我们需要检查解释本身，不能因为结论正确就默认过程正确。

RFT 的局限也逐渐清楚。所有失败都被丢弃，错误类型的信息没有直接进入最大似然目标；纯结果筛选很稀疏；成功样本可能集中在容易题和少数表达模式。我们要保留题型覆盖，避免一轮轮只训练最容易成功的部分。下一步的问题是：能否让不那么成功的尝试也参与比较，并让反馈直接改变策略？

停下来试一试做一轮生成、筛选、训练、重评。报告每题采样数、通过率、过程抽查率和筛后题型分布。若训练数据数量增加而覆盖题目数量下降，解释这会怎样影响下一轮学习。

本节相关资料

• Zelikman 等，2022，STaR Bootstrapping Reasoning With Reasoning。原文链接 。阅读重点：迭代自生成与 rationalization 如何产生新示范。

• Gulcehre 等，2023，Reinforced Self-Training ReST for Language Modeling。原文链接 。阅读重点：生成与改进如何迭代，以及筛选信号从何而来。

• Yuan 等，2023，Scaling Relationship on Learning Mathematical Reasoning with Large Language Models。原文链接 。阅读重点：采样成功与拒绝采样微调之间的联系。

5.3　人类反馈强化学习 RLHF 从偏好到策略更新

现在，小助能给出两份都算对的解答。一份解释清楚，另一份跳过关键条件。二元答案验证器会给它们相同分数，但学生显然更需要前者。我们需要更细的目标：正确之外，还要有用、清楚、符合任务要求。

我们先收集偏好对，让标注者比较同一问题的两个回答。奖励模型（Reward Model，RM）学习一个分数，希望被偏好的回答通常得分更高。常见的 Bradley–Terry（布拉德利与特里）模型把两个奖励分数之差送入 logistic sigmoid（逻辑 S 形函数），得到偏好概率。这个分数只是从数据里拟合出的代理目标，不是 “人类价值” 的完整测量。把问题记为 x、被偏好与不被偏好的回答分别记为 y 的上标加号与减号，偏好概率写为：

$$P\left(y^{+} \succ y^{-} \mid x\right)=\sigma\left(r\left(x, y^{+}\right)-r\left(x, y^{-}\right)\right)$$

符号 “优于” 表示偏好而非严格事实正确，r 是打分函数，σ 是把分数映射到 0 到 1 的 logistic sigmoid 函数。这里用来训练 RM 的是偏好标签。

有了奖励，我们把语言模型当作策略。状态是问题与此前生成的 token，动作是下一个 token，一段回答是一条轨迹。最简单的策略梯度直觉是：提高回报较高轨迹的概率，降低相对较差轨迹的概率。优势函数衡量结果比基线好多少；它不是由 “好样本” 这个名称自动赋予的，而是由回报估计方式产生。

我们先把反馈学习画成一个循环：采样产生当前策略实际会写的回答，评分器提供反馈，优化器据此更新策略，下一轮样本也随策略变化。图 5.1 将 “谁生成、谁评分、谁更新” 分开；具体优势估计

与更新规则会因算法而异。

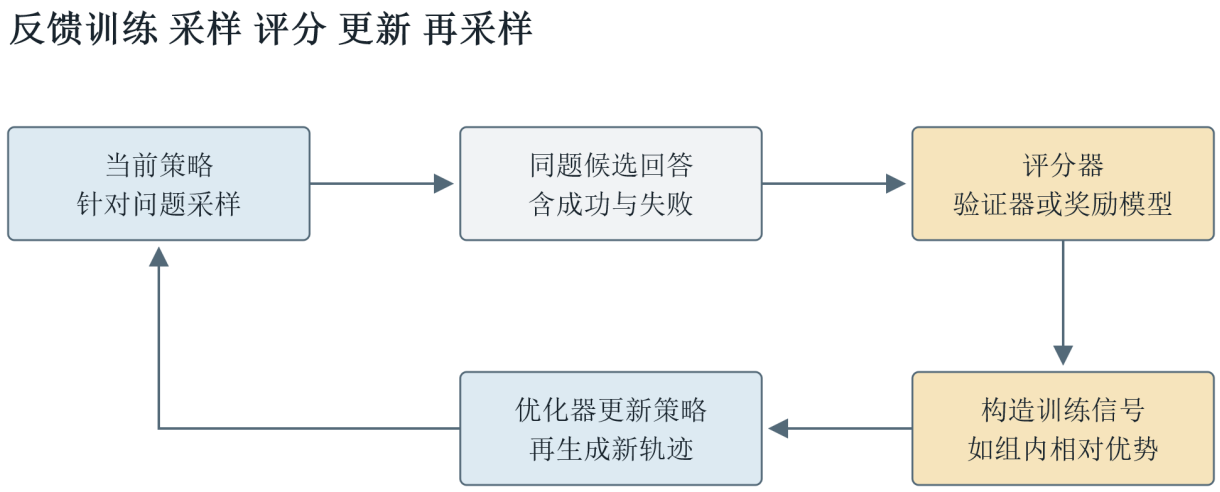


图 5.1 反馈训练的数据流。奖励模型与可执行验证器是两种不同评分来源；图中用组内相对优势作为后续分组比较算法的入口，不表示所有 RL 都必须采用分组比较。

若只追求奖励最大，模型可能找到打分器的偏好捷径，例如冗长、讨好或固定套话。经典的人类反馈强化学习（Reinforcement Learning from Human Feedback，RLHF）因而常加入与参考策略的 KL 惩罚，控制偏离程度。KL 约束降低某些失控风险，却不保证真实质量；参考模型本身也可能有缺陷。令 D 为训练问题分布、π 的下标 θ 为当前策略、π 的下标 ref 为固定参考策略，带 KL 正则且需要最大化的目标写为：

$$J(\theta) = E_{x\sim D,\, y\sim\pi_\theta(\cdot|x)}\left[r(x,y)\right] - \beta\, E_{x\sim D} D_{KL}\left(\pi_\theta(\cdot \mid x) \| \pi_{ref}(\cdot \mid x)\right)$$

期望符号表示对问题与生成回答求平均；第一项提高奖励，第二项扣除偏离参考策略的代价，β 是正的权衡系数。实际使用的是从采样估计的目标与梯度。

近端策略优化（Proximal Policy Optimization，PPO）用概率比和裁剪等机制限制单次更新幅度。这里有两个不能混淆的对象：旧策略用于比较采样时和更新时的概率，参考策略常用于定义偏离惩罚。二者可以来自相同初始检查点，但承担不同角色。PPO 采集当前或近期策略的轨迹，再做若干次更新，因此工程上存在受控的策略滞后，而非每一步都严格同分布。

经典语言模型 PPO 管线常涉及策略、参考模型、奖励模型和价值模型四种角色。它们不一定始终以四个完整独立大模型同时驻留显存，也可以共享部分结构、卸载或分布部署。复杂度来自采样、打分、价值估计、同步和稳定更新共同构成的系统。

如果任务能可靠验证，我们也可以直接用答案正确性或执行结果奖励，不必训练 RM。这通常称为可验证奖励强化学习（Reinforcement Learning with Verifiable Rewards，RLVR）。开放偏好和可验证任务都能使用强化学习；RL 也可以有过程奖励，并非天生只给结尾一次反馈。

停下来试一试为三种奖励写出潜在漏洞：最终答案正确、回答越长越好、评审模型更喜欢。再给每一种配一个独立检查。我们要验证的是真实目标，而不是奖励函数是否忠实奖励了自己写下的规则。

本节相关资料

Ouyang 等，2022，InstructGPT。原文链接 。阅读重点：SFT、偏好奖励与策略优化的接口。

Schulman 等，2017，Proximal Policy Optimization Algorithms。原文链接 。**PPO 的开创性论文**。阅读重点：旧策略概率比、优势与裁剪分别解决什么问题。

5.4　组相对策略优化 GRPO 同题比较如何产生信号

价值模型增加了训练成本。我们于是换一种估计方式：对同一道题采样一组回答，根据组内奖励比较每份回答好多少。这是理解组相对策略优化（Group Relative Policy Optimization，GRPO）的入口。它使用组内相对回报作为基线，省去独立价值评估器（critic）的一种常见设置。

设一组四个回答的奖励是 0、1、1、0，均值为 0.5。直觉上，两个成功回答高于本组平均，两个失败回答低于平均。常见实现还会除以标准差，并加入很小的稳定项。我们先用这个简化优势理解信号，再回到论文中的概率比、裁剪和 KL 项。把第 i 份回答的组相对优势记为 A 的下标 i 上加帽号，这个常见简化估计写为：

$$\widehat{A}_i = \frac{r_i - \overline{r}}{std\left(r_1, \ldots, r_G\right) + \epsilon}$$

这里 r 的下标 i 是该回答的奖励，r 上的横线是组均值，std 是组内标准差，G 是组大小，ε 是防止除零的小正数。

若一组奖励全是零，优势通常全为零，模型没有从组内比较获得哪条更好的信息。若全是一，结果奖励部分也缺少区分。此时其他正则项仍可能产生梯度，因此不能把 “没有相对奖励信号” 说成 “所有参数绝对不变”。我们会记录零方差组比例，判断训练数据是否处在可学习的难度区间。

组内比较的收益伴随额外采样成本。组越大，可能更容易发现差异，但也消耗更多 token；同一题的多次回答高度重复时，收益有限。若按组标准差缩放，题目之间的权重也会改变；不同变体在归一化、长度处理和更新方式上并不相同，不能把所有 “GRPO” 视作一套固定实现。

GRPO 被 DeepSeekMath 使用，并在后续推理模型工作中受到关注。但它没有单独 “创造思维链” 的魔法开关。初始模型、数据难度、奖励可靠性、生成长度与系统实现共同影响结果。省去 critic 能减少一类成本，不能省去对采样与反馈的设计。

回到小助，我们不再只看平均奖励。我们还看每题成功率、组内差异、输出重复率、截断率和长度分布。它们告诉我们奖励曲线背后是否真的存在可用比较，也为接下来两种训练失败提供线索。

停下来试一试分别计算奖励组 0、0、0、0；1、1、1、1；0、0、0、1 的相对优势。思考如何在固定采样预算下选择题目难度。不要直接得出 “删掉所有全对题” 这类脱离覆盖需求的结论。

本节相关资料

Shao 等，2024，DeepSeekMath Pushing the Limits of Mathematical Reasoning in Open Language Models。原文链接 。阅读重点：PPO 到 GRPO 的变化，以及组内基线的定义。

DeepSeek-AI 等, 2025, DeepSeek-R1 Incentivizing Reasoning Capability in LLMs via Reinforcement Learning。原文链接 。阅读重点：奖励、冷启动与多阶段训练如何配合。

5.5　RL 泛化诊断训练涨分而验证不涨

我们完成一轮 RL，训练奖励从教学案例中的 0.35 升到 0.70，开发集答案正确率却几乎不动。此时最危险的反应，是把训练跑得更久。我们要先确认横纵轴究竟是什么。

“训练涨” 可能指在线生成轨迹（rollout）的奖励，也可能指固定训练题的独立评估；“验证不涨” 可能指最终答案正确率，也可能指人工偏好。若前者是代理 RM 分数，后者是真实任务质量，两个指标原本就不等价。我们必须在固定训练题和开发题上，都用同一套解码预算与独立评分重评一次，把测量差异先拿掉。

第一种可能是记住了题目或模板。我们按原始题族切分，测试换数值、换叙述、换组合方式。若只在见过的表述上变好，继续增加同类重复题可能让曲线更漂亮，却不能支持迁移。我们应补充机制相同但表面不同的训练覆盖，并保留真正未参与开发的测试题。

第二种可能是奖励被钻空子。小助把 “因此答案正确” 写得更频繁，RM 更喜欢，独立验证器却看不到改善。我们检查高奖励失败样本，尤其看奖励提升最大的部分。必要时修评分规范、增加反例、改验证器，再从合理检查点重训。KL 加大可能抑制漂移，但它不能修复奖励定义中的根本错误。

第三种可能是训练与开发分布不同。训练主要是一两步代数，开发集更多是长条件、多步组合。训练收益可能真实，但局限于已覆盖区域。我们按知识点和难度分桶，看哪些桶有迁移，再决定补数据、安排课程或调整目标任务。总分不动可能隐藏局部进步，也可能隐藏一个桶上涨另一个桶下降。

第四种可能是评估噪声或预算变化。小评估集、随机采样、答案解析错误，都能掩盖变化。我们报告题目数和重复次数，对同一题做成对比较，必要时估计置信区间。若新模型用了更长输出或更多采样，还需画质量与 token 成本的关系，而不是只报最终分。

我们可以把证据整理为一个紧凑矩阵，再选下一步实验。

观察	优先假设	下一步检查
训练 RM 涨独立正确率不涨	奖励偏离真实目标	人工审查高奖励失败样本
固定训练题涨未见题不涨	记忆或覆盖不足	按题族和难度分桶重评
答案更长得分差异不稳定	预算或测量混淆	固定 token 与采样预算
一个题型涨另一个题型降	收益不均或能力干扰	分项指标与旧能力回归测试

停下来试一试从四种假设中选两种，分别写出 “什么结果会推翻它”。不能被任何观察推翻的解释，通常不足以指导下一次训练。

本节相关资料

Gao 等，2022，Scaling Laws for Reward Model Overoptimization。原文链接 。阅读重点：怎样分别衡量代理目标与更可信的目标。

Srivastava 等，2022，Beyond the Imitation Game Quantifying and extrapolating the capabilities of language models。原文链接 。阅读重点：任务多样性为何影响我们对能力的判断。

5.6　RL 训练诊断从数据与判题检查到补回 SFT

另一轮实验中，小助的训练奖励几乎是一条直线。我们先做一次很小的链路检查：确认官方样例或简单题能运行、参数确实更新、采样端加载了新权重、奖励方向没有写反。使用经过验证的成熟训练基础设施（infrastructure，常简称 infra），并确认关键配置后，我们把主要排查精力转向题目难度、参考答案与判题器。这是当前实验的排查顺序，不是对所有框架故障率的统计结论；有明确异常日志时，仍应沿证据检查实现。

先看数据是否处在可学习的难度区间我们固定采样预算，对每题生成一组回答，查看每题成功率与同分组比例。如果绝大多数题都远超小助当前能力，组内反复全错，GRPO 就缺少有用的相对奖励信号。我们先排除错答案和误判，再适当降低题目难度、补充中间难度题或拆分子任务。全对题也不能不看覆盖需求就全部删除；目标是让当前策略有机会产生可以比较的尝试。

造数据时就交叉核验答案我们不把一个强模型生成的答案直接当作标准答案。先让多个强模型独立解题，尽量不向它们展示候选标准答案，避免互相照抄或被同一答案锚定；再把最终答案按约定格式归一化，比较是否一致。有分歧的题先复核、重写或剔除，不能靠多数票强行入库。模型之间也可能共享错误，所以一致只是筛选信号；能用计算、执行、符号检验或原始资料确认的题，应继续核验，并抽样人工检查一致通过的样本。

我们把用于自动判分的最终答案设计得简短清晰，例如一个词语、一个数值或一个短语，同时保存可核查的解题依据。数值题预先规定单位、精度与容差；词语和短语题规定合理的别名、大小写与等价表达。这里约束的是最终答案与判分接口，推理过程仍可按任务需要保留。题目本身也要避免条件缺失、多解却只登记一个答案等问题。

把判题器当作需要验收的组件我们用已知正确、已知错误及边界样本测试判题规则。例如 “5” “5.0” “a = 5” 是否应当等价，要由这道题的答案规范决定；“5 米” 不能在量纲不符时仅因含有数字 5 就得分。我们检查答案抽取、等价判断、单位、超时和空输出，再抽样复核真实 rollout。假阴性会让有效尝试拿不到奖励，假阳性则可能让模型学会利用判题漏洞；不能只看评分程序是否跑完。

优先构造短答案题，谨慎使用选择题选择题让模型可以通过猜测、排除或选项中的表面线索得分，而不必完成我们希望训练的求解。因此，在能够可靠判分时，我们优先让小助直接给出词语、数值或短语，而不是把训练题默认做成四选一。猜中本身不等于奖励投机，但持续利用选项位置、措辞或解析漏洞，会使奖励与实际求解能力脱离。

如果任务必须使用单选题，我们可以把候选数扩展到 10 项，条件允许时再试 20 项，并保证干扰项有

意义、题目仍有唯一正确答案。在完全均匀盲猜且只有一个正确选项时，命中率分别是 10% 和 5%；这不是实际模型的猜测率保证。无效干扰项容易被排除，增加选项还会增加输入长度与阅读难度，因此要比较这些变化后的真实收益。

在适用时，我们可以为每次 rollout 独立打乱选项顺序，并同时重建显示标签与正确答案映射。稳定保存每个选项的内容标识、排列和随机种子，判题时先把模型选择映射回实际内容，再与标准答案比较。若只打乱选项却沿用原来的字母答案，反而会制造错误奖励；“以上皆是” 等依赖位置的表述也须先改写。顺序随机化能检验和削弱某些位置捷径，但不能消除所有猜测与奖励投机。

排查完数据与判题，再看温度、生成长度、KL 约束、学习率、优势与裁剪：输出是否过度重复，是否被截断，更新是否过弱？我们一次改一个因素。若干净题目和可靠判题都已确认，而小助在可接受预算内仍反复缺少关键步骤，再用经审核的针对性 SFT 示范建立起点。补完以后，先检验留出题，再观察是否出现更多有效尝试与组内差异，最后重启 RL。

我们在示范与反馈之间按证据往返。成熟链路做简短确认，数据难度、答案与判题做重点复核；补 SFT 的依据是有效探索仍缺少必要能力，而不是奖励曲线暂时没有上升。

停下来试一试审查十道自建题，记录难度、多个强模型的独立答案、标准答案证据与判题边界测试。再挑一道选择题，扩充干扰项并生成三种随机排列，核对每种排列都映射回同一正确内容。根据证据决定修题、修判题、调采样，还是补 SFT。

本节相关资料

Shao 等，2024，DeepSeekMath。原文链接 。阅读重点：组内比较为什么依赖可区分的奖励，以及采样、奖励与更新怎样协同。

DeepSeek-AI 等，2025，DeepSeek-R1。原文链接 。阅读重点：可验证奖励、数据与冷启动的适用条件；不要把某个成功配方当作所有任务的固定步骤。

Pezeshkpour 与 Hruschka，2023，Large Language Models Sensitivity to The Order of Options in Multiple-Choice Questions。原文链接 。阅读重点：选项排列如何影响测量；正文中的 10 或 20 项与逐次打乱是需要在具体任务中验证的工程建议，并非该论文给出的通用最优配置。

第六章　偏好优化与同策略蒸馏

6.1　直接偏好优化 DPO 为什么可以绕过奖励模型

我们已经收集了“同一题，两份回答，哪份更好”的数据。PPO 路线先把它们变成奖励模型，再在线采样更新策略。我们自然会问：能否直接用偏好对训练小助，少维护一些组件？直接偏好优化（Direct Preference Optimization，DPO）就从这个问题进入。

先回到带 KL 正则的奖励最大化。若奖励给定，参考策略固定，并满足归一化和支持等条件，最优策略可以写成“参考概率乘上指数奖励，再归一化”。这是分布层面的解析关系，不是整个神经网络训练问题已经有了闭式参数解。用 π 的上标星号表示最优策略、Z 表示归一化因子，解析关系写为：

$$\pi^*(y \mid x) = \frac{\pi_{ref}(y \mid x) \exp\left(r(x,y)/\beta\right)}{Z(x)}$$

这里 β 沿用 KL 权衡系数，exp 表示指数；Z(x) 对问题 x 的所有回答候选求和，使右侧仍为合法概率分布。闭式关系中的“最优”依赖前述假设。

我们把它反过来整理，奖励可以表示成策略与参考策略的对数概率比，再加一个只依赖问题 x 的项。拿同一道题的两个答案相减时，这个项消失。于是，偏好概率能直接写成策略概率的函数，不必另训显式奖励网络。对上式取对数并整理，把奖励改写成策略概率比，得到：

$$r(x,y) = \beta log \frac{\pi^*(y \mid x)}{\pi_{ref}(y \mid x)} + \beta log Z(x)$$

β log Z(x) 只依赖问题，不依赖回答。因此，同一道题的两个奖励相减时，这一项恰好抵消；不是在所有计算中都可以直接丢掉它。

为了看清损失，我们定义一个差值 Δ：被偏好回答相对参考的对数概率增量，减去不被偏好回答的对应增量。若 Δ 越大，模型就越符合这条偏好。DPO 用负对数 sigmoid 拟合标签。沿用同一问题 x 与正负回答的记号，差值 Δ 的定义为：

$$\Delta_\theta = log \frac{\pi_\theta\left(y^+ \mid x\right)}{\pi_{ref}\left(y^+ \mid x\right)} - log \frac{\pi_\theta\left(y^- \mid x\right)}{\pi_{ref}\left(y^- \mid x\right)}$$

每个比值都把当前策略与同一个参考策略比较；Δ 先计算正回答的相对增量，再减去负回答的相对

增量。

再把这个差值代入偏好概率，对偏好数据取负对数似然，DPO 损失写为：

$$L_{DPO} = -E_{(x,y^+,y^-)} log\sigma\left(\beta\Delta_\theta\right)$$

这里的期望对偏好数据中的三元组取平均，σ 与上一章相同，β 调整差值进入偏好模型的尺度。训练仍然通过梯度更新神经网络参数。

这里用的是回答序列的概率，实际通过 token log probability 加总计算，通常不把提示文本（prompt）对应的 token 计入回答概率。长短答案、截断、补齐掩码（padding mask）和模板仍会影响结果。它与 SFT 一样可以 teacher forcing 计算给定回答的概率，但损失还比较正负回答及参考模型，不能直接说成 “SFT 换个名字”。

我们把四次概率计算画在图 6.1：两份回答分别经过当前策略与冻结参考策略。先减参考，再比较正负，才能得到前面定义的 Δ。图里没有在线采样箭头，对应本节的固定偏好数据设置。

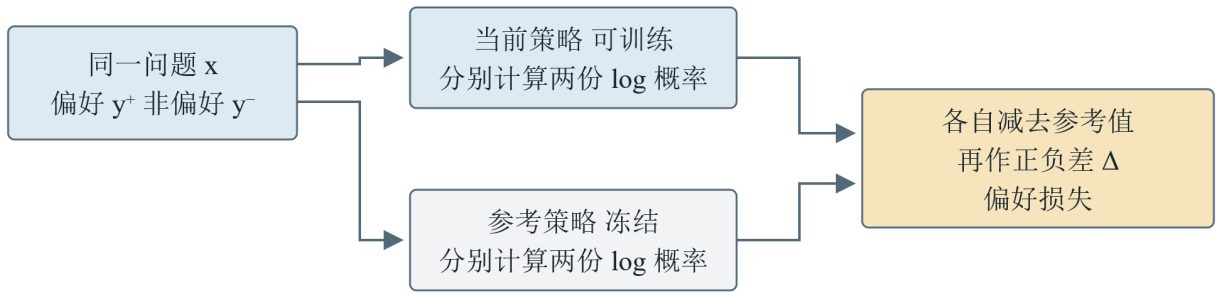


图 6.1 DPO 的比较对象。参考策略参与计算但不随本次损失更新；梯度只更新当前策略。离线数据是否覆盖当前行为，需要在下一节另行检查。

我们用一个小例子观察参考的作用。假如训练策略从参考初始化，最初每份回答的对数概率比都为零，Δ 也为零，对这对样本预测偏好概率为 0.5。训练再根据标签拉开相对差异。若标签总偏爱长答案，模型也可能学到长度捷径；省掉显式 RM 并不自动省掉偏好数据中的偏差。

DPO 的理论联系建立在偏好建模与理想化优化条件上。有限数据、受限模型族和分布外输入，会使实际行为偏离理想。我们获得的是一种直接优化方式，仍然需要训练和独立评估。“闭式解” 解释了重参数化，不意味着运行一次公式就能得到最终权重。

停下来试一试假设某条偏好数据中，两个回答的 “相对参考对数增量” 分别为 0.4 和 0.1，计算 Δ。再将偏好标签反过来，解释损失会鼓励怎样的变化。想一想错误偏好为什么会造成有方向的损害。

本节相关资料

• Rafailov 等，2023，Direct Preference Optimization Your Language Model is Secretly a Reward Model。原文链接 。阅读重点：从最优策略关系反解奖励，以及为什么归一化项在偏好比较中抵消。

• Azar 等，2023，A General Theoretical Paradigm to Understand Learning from Human Preferences。原文链接 。阅读重点：偏好建模和优化目标的假设怎样影响方法选择。

6.2　离线与在线偏好优化数据如何跟上策略

我们用同一份偏好集做了许多轮 DPO。训练集的偏好分类越来越容易，实际对话却出现一类新错误：小助在较长的解释中反复自我肯定。旧偏好对里没有这种回答，我们也从未评价过它。此时，问题从 “如何拟合标签” 转为 “标签覆盖了谁的行为”。

离线偏好优化把已有回答当作固定训练材料。我们不需要在每个优化步骤在线生成，成本容易控制，数据也便于审核。可当策略明显变化，静态样本未必覆盖新模型常生成的轨迹。这里讨论的分布覆盖与 teacher forcing 的前缀错位有关，却不是完全相同的问题；我们不把所有离线局限都统称 exposure bias。

我们可以周期性让当前小助生成新候选，再标偏好、再优化。这是在线或迭代偏好优化的一种思路。它把反馈追到学生的新行为上，但也增加采样、评审与版本管理成本。一次收集、多轮训练与持续重采样之间存在很多中间方案，并非只有 “全 on” 或 “全 off” 两个格子。

阅读变体时，我们继续用三个问题定位：信号是什么，数据由谁产生，更新依赖哪些组件。恒等偏好优化（Identity Preference Optimization，IPO）研究另一类偏好目标；卡尼曼与特沃斯基优化（Kahneman–Tversky Optimization，KTO）可以使用好坏标签而不要求每条都是成对偏好；简单偏好优化（Simple Preference Optimization，SimPO）以平均序列对数概率构造无参考奖励；优势比偏好优化（Odds Ratio Preference Optimization，ORPO）把监督学习与偏好约束结合，省去独立参考模型。知道这些差别，就能理解它们各自减少了什么成本，而不急着背公式清单。

我们也提醒自己：无参考并不等于不会漂移；在线并不等于标签可靠；损失更简单也不等于任务一定更强。方法比较应固定起点、偏好来源、生成预算和评价协议。若一个方法用了更强老师重新生成数据，它的收益不能全部算到损失函数头上。

我们会先选实现清楚、数据适配的方法建立基线。当独立评估显示静态数据覆盖不足，再引入迭代收集。这个顺序使每一次复杂度增加都对应已观察到的需求。

停下来试一试为小助的新错误写一对偏好回答，控制长度和语气，只改变关键缺陷。再说明：这条数据应该来自旧模型、当前模型，还是老师模型？不同来源改变了什么？

本节相关资料

• Azar 等，2023，A General Theoretical Paradigm to Understand Learning from Human Preferences，包含 IPO。原文链接 。阅读重点：为何要重新检查偏好目标。

• Ethayarajh 等，2024，KTO Model Alignment as Prospect Theoretic Optimization。原文链接 。阅读重点：不成对的好坏标签怎样提供信号。

• Meng 等，2024，SimPO Simple Preference Optimization with a Reference-Free Reward。原文链接 。阅读重点：参考模型与长度处理的改变。延伸比较 ORPO：原文链接 。

6.3　同策略蒸馏 OPD 让老师反馈学生轨迹

小助的很多错误发生在中途。单个最终奖励告诉它“没有做对”，却未必指出从哪个分叉开始偏离；照抄老师的完整解答又主要覆盖老师走过的前缀。我们于是尝试另一种教学安排：学生先自己走，老师在学生走到的位置上给反馈。

这就是同策略蒸馏（On-Policy Distillation，OPD），也称在策略蒸馏。它不是 2025 年才突然出现的单一算法；广义知识蒸馏（Generalized Knowledge Distillation，GKD）与 MiniLLM 等工作已经研究过相关思路；MiniLLM 是方法名称，这里不另造缩写。不同实现可以使用不同散度、不同轨迹混合与不同梯度估计。我们先理解共同结构，再选一种反向 KL 形式作例子。

在学生生成的前缀 h 上，老师和学生各给出下一 token 分布。反向 KL 把学生分布放在期望的采样侧：学生常选择、老师却不支持的 token，会产生较大的平均代价。若能取得完整分布，就可以对词表求和。把老师策略记为 π 的下标 T，在同一个学生前缀 h 上，反向 KL 定义为：

$$D_{KL}\left(\pi_\theta \| \pi_T \mid h\right)=\sum_v \pi_\theta(v \mid h) \log \frac{\pi_\theta(v \mid h)}{\pi_T(v \mid h)}$$

求和的 v 遍历词表，θ 表示学生参数；两边必须在同一前缀下给概率。该式是词表分布的 KL，不是单个 token 的人类正确性标签。

若只查询学生实际采样 token 的 log probability，可以用对数概率差构造随机估计，并把相反方向的差作为逐 token 反馈。单个采样差值可以为负，KL 的非负性指的是分布期望；“红色 token” 也不自动等于人类语义上确定错误的步骤。把第 t 步学生前缀记为 h 的下标 t、采样动作记为 a 的下标 t，用老师与学生的对数概率差构造反馈：

$$\hat{r}_t=log\pi_T\left(a_t \mid h_t\right)-log\pi_{student}\left(a_t \mid h_t\right)$$

π 的下标 student 表示用于反馈计算的学生策略，帽号提示这是基于采样的信号。实际实现要说明使用采样时还是更新时的概率，以及哪些量停止梯度。

学生已经走错时，老师面对的是错误前缀。老师可能纠正它，也可能只给出在错误前提下自然的延续。于是，分布反馈很密集，却不等于可靠的步骤真值。逐 token 的局部更新、完整序列 KL 和带未来回报的策略梯度之间，也需要具体说明估计器与梯度处理；不能把任意逐 token 对数差实现都说成严格相同的全序列优化。

我们把两个 KL 方向放回同一幅图景。正向 KL 从老师分布出发，通常更强调覆盖老师的多种模式；反向 KL 从学生分布出发，在受限模型族等条件下常表现出更强的模式选择倾向。这是有条件的行为直觉，并非“正向总平均、反向总最好”的定律。数据在哪些前缀上采集，与散度方向同样重要。

Thinking Machines 在 2025 年的研究博客中展示了推理与个性化的 OPD 实验，并用采样 token 的老师 log probability 构造密集反馈。我们把它视作有用的实验案例，不把其中效率结果写成跨任务的固定倍数。低 KL 说明学生接近老师，不能保证接近事实真相；老师的错误、风格偏差与盲点仍会传递。

回到续训退化，我们可以让学生在真实任务前缀上向保留良好行为的老师学习。但老师若没有新课程知识，就不能指望它凭空教会学生。实际还要考虑 tokenizer 对齐、概率接口是否可用、老师计算成本与数据覆盖。OPD 把反馈带到学生路径上，成功条件仍要通过实验确认。

停下来试一试让学生先写出一个错误首步，再分别比较老师“续写该前缀”和“批改该前缀”的回答。解释为什么两种条件下的概率信号未必具有同样含义。若老师只提供文本、不给 log probability，还能做序列蒸馏，但它与这里的逐 token 概率反馈不是同一接口。

本节相关资料与研究材料

• Agarwal 等，2023，On-Policy Distillation of Language Models Learning from Self-Generated Mistakes，GKD。原文链接 。阅读重点：学生生成的数据与不同散度目标。

• Gu 等，2023，MiniLLM。原文链接 。阅读重点：反向 KL 与自回归蒸馏中的优化问题。

• Kevin Lu 与 Thinking Machines，2025，On-Policy Distillation，研究博客。原文链接 。阅读重点：采样 token 反馈的实现与实验条件；它不是普适效率保证。

6.4　后训练方法比较依据信号与分布作选择

我们现在可以把方法放回同一张地图。它们并非从旧到新的单向升级链，适合哪条路线取决于缺陷和可获得的反馈。

我们用三个轴定位：信号从哪里来，轨迹由谁产生并多久更新，优化需要维护什么组件。信号成本也不一定单调下降，老师前向、人工标注与规则验证的价格取决于场景。

方法	主要训练信号	轨迹或数据来源	更新方式
SFT	目标回答 token	人类或老师等外部示范	最大似然
拒绝采样 RFT	筛过的成功回答	当前或旧检查点采样	筛选后最大似然
PPO 类 RL	结果或过程回报	当前或近期策略采样	带约束的策略梯度
GRPO 类 RL	组内相对回报	同题多次策略采样	相对优势更新
离线 DPO	被偏好与不被偏好回答	固定偏好数据	直接偏好损失
OPD	老师概率或分布	学生轨迹或混合轨迹	散度或相应梯度估计

我们据此决定下一步。若新行为连一条可靠示范都没有，先建设示范；若成功路径可采到且可验证，

比较 RFT 与 RLVR；若有明确偏好对但在线成本受限，建立 DPO 基线；若有合适老师和可用概率接口，再考虑 OPD。若问题只是缺少最新课规，回到资料检索，不必先更新权重。

停下来试一试为 “不会输出指定的结构化格式” “答案偶尔正确” “奖励一直为零” “不知道本周作业截止时间” 各选一个优先行动，并写出采用该行动前必须确认的证据。不要让四个问题都得到同一个 “继续训练” 的答案。

本节相关资料

• Shao 等，2024，DeepSeekMath。原文链接 。阅读重点：把多种后训练方法放进共同分析框架。

• Rafailov 等，2023，DPO。原文链接 。阅读重点：目标联系不等于工程路径相同。

第七章　推理策略与推理时计算

7.1　思维链 CoT 与推理时计算

训练告一段落，我们给小助一道需要多步转换的题。让它直接报答案时容易出错；让它先列出条件、写中间计算，再给结论，成功率可能提高。我们没有更新权重，却给了它更多生成过程。这就是思维链（Chain-of-Thought，CoT）与推理时计算（test-time compute）的入口。

自回归模型每生成一个 token，都进行一次依赖此前内容的计算。中间文字可以记录变量、拆解子问题、保留暂时结果，因此延长序列有时相当于把计算展开到更多步骤。但可见解释不是对内部机制的完整读出，模型也可能先偏向一个答案，再写出似乎合理的说明。我们评估步骤是否有效，不把流畅的长解释当成真实正确的证明。

我们还可以并行多试几次，再用选择器挑选。若题目有可规范化的最终答案，可以多数投票，这就是 self-consistency 的常见用法。它利用不同采样路径在答案上的一致性，却要求正确答案能形成足够支持。若模型反复犯同一种系统错误，多数投票反而会稳定选错。

另一种方式是多候选择优（best-of-N）：生成 N 个候选，由验证器或奖励模型选最好的。投票不需要评估每条步骤质量，选择器则可以利用更多信息，但会受到自身误差影响。若训练奖励曾被钻空子，我们不能未经验证就把同一个打分器当成完美选择器。

我们把三种方案放在相同预算下比较：一条长解答、几条短解答、多候选加验证。难题可能值得更多计算，简单题则无需长篇推导。若把更长输出全部视为进步，就会忘记用户需要的是及时而可靠的帮助。真正的曲线应当是任务质量相对于总 token、延迟和计算成本的变化。

训练 scaling 改变模型能力与分布，推理时计算则在给定模型上分配更多尝试或过程。它们可以互相补充，也可能受到不同瓶颈限制。模型若缺少必需知识，多想一会儿未必能补出来；若只是算术容易失误，调用计算器可能比再写一千 token 更有效。

停下来试一试固定总生成预算，比较单条长回答、四条短回答投票和四条短回答加验证器。分别报告正确率、无效格式比例和平均延迟。解释为何不能只对比 “单次短答” 与 “无限制多次长答”。

本节相关资料

• Wei 等，2022，Chain-of-Thought Prompting Elicits Reasoning in Large Language Models。原文链接 。阅读重点：示例中的中间步骤如何影响任务表现。

• Wang 等，2022，Self-Consistency Improves Chain of Thought Reasoning in Language Models。原文链接 。阅读重点：多样采样与答案聚合的分工。

• Snell 等，2024，Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters。原文链接 。阅读重点：推理预算如何随问题难度分配。

7.2　推理模型从示范和结果反馈到蒸馏

观察到“多写步骤有时有用”，不等于模型已经学会稳定推理。小助可能把每题都写得很长，却重复同一句话；也可能在关键处跳步。我们要训练的是有效的推理行为，而不是某种看起来像思考的外观。

示范路线提供高质量解题轨迹，让学生学习分解、求解和检查；反馈路线让模型自己尝试，再依靠可验证结果或过程评价改进；蒸馏路线把强模型的解题数据或分布反馈交给较小学生。它们可以串联，也可以在不同阶段交替。对一个薄弱起点，先给有效示范常能改善后续探索；对已有丰富能力的起点，直接 RL 也可能发现新的有效行为。

DeepSeek-R1-Zero 的实验探索了从基座直接进行大规模 RL 的路线；R1 则使用冷启动数据和多阶段训练，处理可读性、语言一致性等需求。我们阅读时区分研究问题：前者展示特定起点与奖励条件下直接 RL 的可能性，后者展示面向更完整助手行为的组合管线。两者都不能被压缩成“只需 GRPO 就会自动推理”。

过程监督可以更细地评价中间步骤，结果监督则通常更容易自动获得。前者需要可靠的步骤标签或评审，后者可能把偶然猜中当成成功。我们可以抽查“答案正确但理由无效”和“中间合理但最后算错”两类样本，理解不同信号各自漏掉什么。

小模型学习强模型时，也有两种不同蒸馏。把老师完整解答做成数据再 SFT，是离线序列蒸馏；让学生自己生成、再查询老师分布，是第六部分的 OPD。长推理过程蒸馏并不天然等于 OPD。学生若模仿了复杂措辞却无法执行其中步骤，说明数据难度、表示能力或优化还需要调整。

我们还会遇到 thinking 与 non-thinking 模式。具体角色标记、开关和停止规则由模型定义，不能把某个产品接口当成所有模型的标准。实验时记录实际模板、是否允许生成推理段、推理预算与答案抽取方式；否则同一个模型也可能因为入口不同呈现完全不同的表现。

停下来试一试让学生模仿一份过长且跳步的老师答案，再给一份更短但关键步骤完整的答案。先人工比较可学性，再决定如何制作训练集。老师答得对，不代表任何写法都适合当前学生学习。

本节相关资料

• DeepSeek-AI 等，2025，DeepSeek-R1。原文链接 。阅读重点：区分 R1-Zero、冷启动、RL 和离线蒸馏的具体作用。

• Lightman 等，2023，Let’s Verify Step by Step。原文链接 。阅读重点：过程监督与最终结果监督的差别。

• Agarwal 等，2023，GKD。原文链接 。阅读重点：为什么老师轨迹与学生轨迹会给出不同训练问题。

7.3　KV 缓存与推理服务长上下文的资源成本

小助终于能认真解题，却让服务器变慢了。我们没有改参数，显存仍然迅速增长。原因之一是每个会话在生成过程中保留了越来越长的键值缓存（Key–Value Cache，KV Cache），后文简称 KV 缓存。

推理可分为预填充（prefill）和逐步解码（decode）。prefill 处理输入前缀，计算各层 key 与 value；decode 每轮生成一个新 token，查询此前缓存。缓存避免反复重算所有历史位置的 K、V，但不让历史读取变成零成本。随着上下文增长，每个新 token 需要处理的历史信息通常也更多。

图 7.1 接回第二部分的 QKV：进入新位置后，我们计算这个位置的新 Q、K、V；新 Q 读取缓存中的历史键值以及当前位置键值，完成注意力，再将当前位置 K、V 追加进缓存。历史位置在因果模型中不受后续 token 影响，因此相同前缀下可复用这些历史状态。这里画的是逐位置的数据依赖，具体算子可以把追加与读取融合执行。

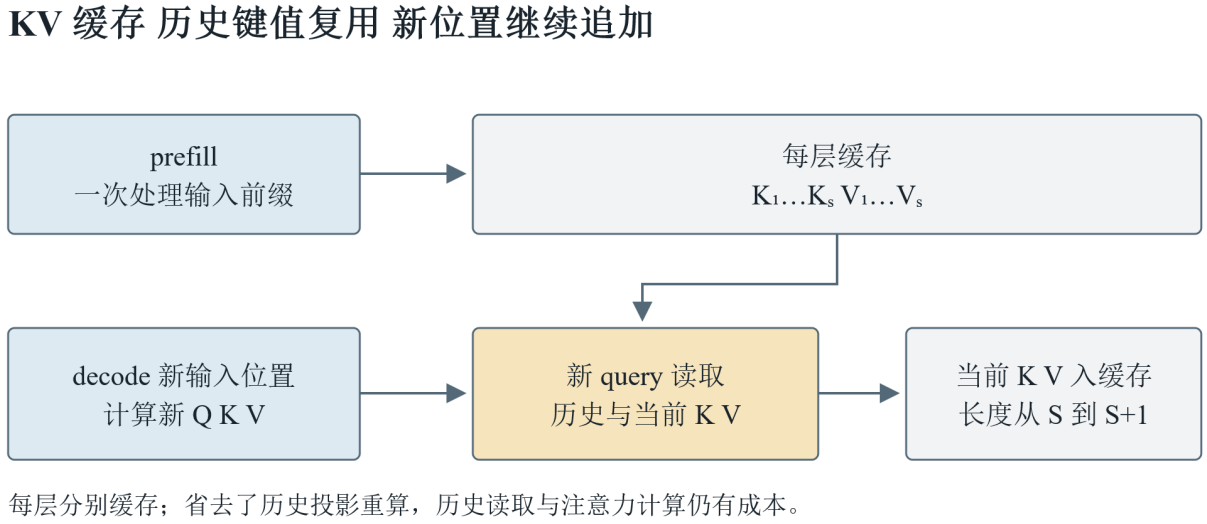


图 7.1 从 prefill 到 decode 的键值复用。每一层分别保存缓存；索引表示已经处理的输入位置，并不表示提前知道下一次生成的 token。

对一个简化的稠密注意力模型，单序列缓存占用可按两份张量、层数、KV 头数、头维度、序列长度与每元素字节数相乘估算。这里用 KV 头数而不是必然使用 query 头数，因为分组查询注意力（Grouped-Query Attention，GQA）等架构可能让多个 query 头共享 K、V。用 M 表示字节占用，L 表示层数、H 的下标 KV 表示 KV 头数、d 表示头维度、S 表示缓存长度、b 表示每元素字节数，估算式为：

$$M_{KV} \approx 2\,L\,H_{KV}\,d\,S\,b$$

前面的 2 对应 key 与 value 两份缓存；该式按一个序列计算，多会话还要考虑并发，以及架构和缓存实现的差异。

例如 32 层、8 个 KV 头、每头 128 维、4096 个 token、每元素 2 字节，单序列 KV 约为 512 MiB。MiB 是二进制兆字节（mebibyte），1 MiB 等于 1,048,576 字节。这个教学估算还没算权重、临时缓冲、分配碎片和多个并发请求；实际系统也可能使用量化缓存、滑动窗口等不同机制。因此，我们把它当作量级检查，不当作完整容量规划。

FlashAttention 通过更好的计算与存储访问方式降低注意力的内存开销；PagedAttention 一类方案改善服务时的缓存管理。它们优化的是不同环节，也不等于任意长上下文都免费。我们观察首 token 延迟、每秒生成 token、并发吞吐和峰值显存，区分用户等待与服务器总效率。

部署还涉及权重量化、批处理和推测解码等方法。我们先了解它们各自在压缩权重、摊薄计算或减少主模型串行调用中的作用，再按实际瓶颈验证。量化后也要重跑关键题和模板测试，不能只检查模型是否成功加载。

从这里回看训练选择，长推理轨迹不仅增加训练数据与 rollout 成本，也增加每次使用成本。我们希望小助在需要时多算，在简单题上及时收束。推理能力与服务效率最终会在同一张成本质量图上相遇。

停下来试一试用上式计算上下文从 4096 增到 8192 后的缓存量，再估算八个同时活跃会话。注明没有计算哪些开销。然后测一次真实服务，看估算与实际相差在哪里。

本节相关资料

• Dao 等，2022，FlashAttention Fast and Memory-Efficient Exact Attention with IO-Awareness。原文链接 。阅读重点：计算复杂度与存储访问成本为什么要分别考虑。

• Ainslie 等，2023，GQA Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints。原文链接 。阅读重点：KV 头共享怎样影响缓存。

• Kwon 等，2023，Efficient Memory Management for Large Language Model Serving with PagedAttention。原文链接 。阅读重点：多请求场景下的缓存组织。

第八章　检索工具与系统评估

8.1　检索增强生成 RAG 让回答获得外部依据

小助能解方程，却不知道“本周作业最晚何时提交”。这不是我们希望通过长推理猜出来的答案。课程规则在变化，部分信息又是私有的，我们决定让小助先找资料，再根据证据回答。

RAG 的核心是把检索到的外部材料纳入生成条件。参数里的知识依然有用，但这次回答的具体依据来自可更新的资料库。RAG 可以改善信息可得性和来源追踪，却不自动保证没有幻觉：检索会漏，资料会错，模型也会错误解读或引用。

我们先把课程文件切成段落。固定长度很简单，但可能把“允许迟交”和“超过多久不计分”拆开。我们结合标题、段落、表格与适度重叠，让每个 chunk 保有必要上下文，同时保存文档名、版本、页码或段落编号。重叠过多会增加重复检索，切得太大又会把相关句子淹没在噪声里，因此要用查询与证据覆盖率选择。

接着，我们用第三部分的嵌入模型编码文档，建立索引；收到查询后编码问题，召回一批候选。若课程名称或作业编号很关键，可以同时做最佳匹配 25（Best Matching 25，BM25）等词法检索，再融合结果。向量擅长语义联系，词法方法对精确字符串常有优势，二者是否互补仍要看任务。

召回后，我们可用交叉编码器（cross-encoder）重排。双塔嵌入先把查询和文档分别编码，便于大规模比较；cross-encoder 让查询与候选一起进入模型，进行更细的相关性判断，代价也更高。我们因此用它处理有限候选，而不是直接对整个资料库逐条运行。

最后，我们把少量证据与问题一起交给小助，要求回答中标明依据，证据不足时说明缺口。资料是待阅读的数据，不是有权改写系统规则的指令。系统还应在检索前执行访问权限约束，避免把用户无权访问的内容放进上下文。模型再聪明，也不能替代这层权限控制。

图 8.1 将离线建库与在线回答分成两条路径。离线索引保存可检索的表示与来源，在线过程才根据这次问题选择证据。最后的核查不是一句“请保证正确”，而是逐条判断答案的陈述是否得到证据支持。

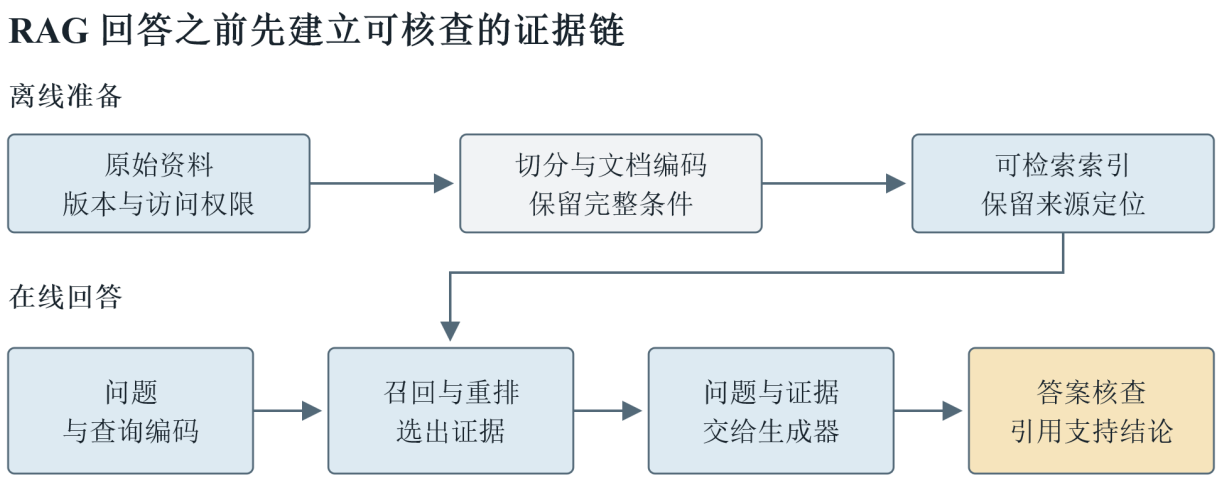


图 8.1 RAG 的工程证据链。资料版本、访问权限和来源定位贯穿流程；正确证据交给生成器后是否仍答错，是下一节诊断的关键分叉。

我们完成的不是一条“向量库接聊天框”的捷径，而是一条可逐段检查的证据链：正确材料存在吗，切分保留了吗，召回找到了吗，重排留下了吗，生成忠实使用了吗？下一节就沿这条链定位失败。

停下来试一试用二十条自建课程规则做最小 RAG。其中一条保留两个版本，一条明确未提供答案。检查系统是否选择有效版本、正确引用，以及是否在未知问题上停止猜测。

本节相关资料

• Lewis 等，2020，Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks。原文链接 。阅读重点：参数化知识与外部检索如何结合。现代工程 RAG 并不都照搬原论文训练结构。

• Nogueira 与 Cho，2019，Passage Re-ranking with BERT。原文链接 。阅读重点：联合编码查询与文档为什么适合重排。

8.2　RAG 诊断沿证据链定位失败

我们问“作业允许迟交几天”，小助引用了一段相关资料，却答错限制条件。只看最终答案，我们不知道该换嵌入模型、扩大上下文，还是重训生成器。我们于是为每个失败保存完整过程：查询、候选、分数、选入上下文的证据和最终回答。

第一步做 oracle context 实验：直接把人工确认的正确证据交给生成器。如果这时仍答错，优先检查阅读、提示、冲突处理或任务能力；如果这时答对，问题更可能在前面的检索与组织。这个实验不证明生成器在所有输入上无错，但可以快速分开两大类瓶颈。

随后逐段检查。正确证据不在资料库，是信息缺失；原文有而 chunk 不完整，是切分问题；穷举相似

度能找到、ANN 找不到，是索引召回问题；召回中有、重排后丢了，是重排问题；上下文中有、答案仍歪曲，是生成使用问题。我们一次只替换一个组件，避免把全部管线重写后失去解释。

若查询太短或术语错位，可以尝试查询改写。假设文档嵌入（Hypothetical Document Embeddings，HyDE）先生成一个假设性文档，再用它做检索桥梁；假设文档的内容可能并不真实，它的用途是帮助定位相关材料，不能作为回答证据。我们始终以实际检索到的原始资料为依据。

若用户问的是整套课程材料的共同主题，单段检索可能不够。图检索增强生成（Graph Retrieval-Augmented Generation，GraphRAG）等方法尝试把实体、关系或社区摘要组织起来支持更全局的问题，但增加了建图、更新和评估成本。若需要多轮查找，可以采用智能体式检索增强生成（agentic RAG），让系统根据已有结果继续检索；每多一步也增加偏航与延迟的可能。

有同学会提议直接放完整资料进长上下文。对于规模可控、需要跨段联系的资料，这可能是好基线；但上下文容量不等于稳定利用能力，长输入还增加成本。我们实际比较 RAG、长上下文和二者组合，而不是先选一个时髦名词。

需求	优先建立的基线	关键验证
少量完整资料的综合阅读	长上下文直接阅读	位置变化与跨段整合
经常更新的大型资料库	RAG	召回和引用支持度
稳定改变输出行为或格式	SFT	新行为与旧能力回归
新知识与新行为同时需要	检索与训练组合	分别测信息获取和行为执行

停下来试一试对五个 RAG 失败分别标记最早出现错误的环节。为其中一个做 oracle context 对照，再决定最小修复。若没有保存中间结果，我们就缺少了最便宜的诊断证据。

本节相关资料

• Gao 等，2022，Precise Zero-Shot Dense Retrieval without Relevance Labels，HyDE。原文链接 。阅读重点：假设文档如何用于检索，而非充当事实来源。

• Edge 等，2024，From Local to Global A Graph RAG Approach to Query-Focused Summarization。原文链接 。阅读重点：全局问题与局部段落检索的差别。

• Liu 等，2023，Lost in the Middle How Language Models Use Long Contexts。原文链接 。阅读重点：放得进去与用得好为什么不同。

8.3　提示与智能体从表达要求到工具调用

学生提出一道带小数的复杂计算。小助会解释思路，却在最后一位出错。继续收集千条同类训练数据之前，我们先问：能否把计算交给可靠工具，让模型负责理解任务与组织结果？

我们先设计清楚的提示。system 说明任务规则，user 提供问题，assistant 产生回答；这些角色要经过对应 chat template 才成为模型输入。few-shot 示例展示所需行为，明确输出结构规范（schema）便于

程序检查。提示不更新参数，却改变条件上下文，因此可以明显改变回答分布。它能唤起已具备的行为，也可能因含糊、冲突或上下文过长而失效。

工具调用（function calling）进一步把输出解释为动作。小助产生工具名和参数；常见载体是 JavaScript 对象表示法（JavaScript Object Notation，JSON）一类结构化格式。外部程序验证 schema、检查权限、执行并返回结果，模型再根据结果继续。结构正确是第一道门槛，参数是否符合任务、工具是否真正执行、结果是否被忠实使用是后续门槛。生成一段看起来像 JSON 的文本并不等于动作已经发生。

推理与行动方法（Reasoning and Acting，ReAct）帮助我们把推理与行动交替组织。我们在实现中保存任务状态、工具返回和关键约束，让系统在观察新证据后选择下一步。这里不必把模型所有内部推演展示给用户；外部可审计记录应包含动作、输入、返回、错误与决策结果。用户需要可检查的进展与依据。

我们也不把工具调用能力全部归功于某一种 SFT。提示示例、监督数据、执行反馈、合成训练都可能帮助形成该行为。对应用开发者，更直接的问题是接口是否支持、参数是否可靠、失败后能否恢复。复杂智能体（agent）框架不能替代这些基础能力。

第一次实验，我们只提供两个工具：检索课程规则与计算器。系统设置最大步数、超时与重试上限，要求在工具失败时说明状态。对于会产生现实写入的动作，由应用执行清楚的权限与必要确认；模型不能因为读到一段资料里的命令就获得额外权限。外部内容与系统指令的边界，也是任务可靠性的一部分。

我们发现长任务还有上下文管理问题。工具返回太长，旧约束被挤掉；摘要太激进，关键条件丢失。我们保留原始日志和结构化状态，在压缩上下文时验证必要字段。一个可恢复的小循环，往往比一开始追求自主完成几十步更适合学习。

停下来试一试给小助接上两个只读工具，测试正常调用、参数错误、工具超时和返回无结果四种情况。逐步记录它是否重试、是否编造成功、是否能停止。成功路径之外，恢复路径同样需要设计。

本节相关资料

• Yao 等，2022，ReAct Synergizing Reasoning and Acting in Language Models。原文链接 。阅读重点：动作与外部观察怎样改变后续决策。

• Schick 等，2023，Toolformer Language Models Can Teach Themselves to Use Tools。原文链接 。阅读重点：工具使用训练可以怎样构造数据与反馈。

8.4　分层评估怎样证明系统真的变好

从最初续写题目到现在，小助已经经历多种训练，也接上资料和工具。如果此刻只展示几个成功演示，我们仍然没有证明它可靠。我们回到第一节的验收约定，把模型能力和系统能力分别测清楚。

模型层评估包括固定题目的答案、解释和指令遵循；检索层包括相关证据是否召回、重排是否保留；系统层还要看工具执行、引用支持、延迟与失败恢复。一个系统得分下降，可能是模型变差，也可

能是资料版本或工具接口变化。分层测量让我们知道变化发生在哪里。

我们把常用基准当成观察窗口，而不是能力全貌。知识问答、数学、代码和人工偏好各测一部分；语言与任务分布不同，也可能改变结果。训练过程中若反复使用某个公开基准做选择，就更需要独立的自建题集确认迁移。我们保存数据来源和去重规则，检查是否意外含有评测答案。

开放回答可以做成对盲评，随机交换左右位置，允许平局，并给出清楚的质量维度。模型评审能降低成本，但可能偏爱长度、特定风格或与自己相近的回答。我们用一部分人工标注校准评审，检查不同子任务的误差。一个评审器总体相关不错，也可能在最重要的失败类型上不可靠。

统计上，我们报告分母与变化幅度。假如只有二十道题，多答对一道就变化五个百分点，不能过度解读。我们可以对同题的新旧结果做成对分析，用 bootstrap 等方法估计不确定性；若多次尝试后只挑最好结果展示，还要承认开发选择带来的乐观偏差。

我们最终选部署版本时，会同时考虑正确、可靠、可解释与成本。旧讲题能力不能因为新课规问答变好就消失；更长回答带来的分数，也要与时延比较。没有一个总分能自动替我们决定所有取舍，评价权重来自实际任务需求。

当新的失败到来，我们把它保存成带标签的案例，先确认属于信息缺失、行为偏移、探索不足、奖励偏差还是系统故障，再决定更新数据、训练或应用组件。这样，评估不只是结课考试，而成为系统继续学习的入口。

停下来试一试设计一页评估报告，至少包含样本数、模型版本、固定解码配置、分项指标、成本和三个失败案例。遮住模型名称，请同学判断是否足以支持“值得替换旧版本”的结论。

本节相关资料

• Zheng 等，2023，Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena。原文链接 。阅读重点：模型评审的偏差与人类偏好评估的设计。

• Chen 等，2021，Evaluating Large Language Models Trained on Code。原文链接 。阅读重点：执行评估与采样预算应怎样共同报告。

8.5　实践路线六次连接全书的实验

概念只有经过操作，才会变成可用的判断。我们把前八部分内容整理为六次实验，每次交付一个能解释的结果。算力不足时，我们可以使用更小模型、较少数据、现成检查点或离线日志，保留实验逻辑，不必为了规模跳过诊断。

第一次实验，我们训练一个玩具自回归模型。使用少量明确来源的文本，先划分训练与验证，再选字符级或现成 tokenizer，记录固定提示的生成变化。验收不是“它会写好文章”，而是我们能解释输入、目标、因果掩码与损失如何连接，能发现过拟合与数据问题。

第二次实验，我们建立一个小型课程资料库。用同一嵌入模型完成去重候选、聚类和检索，人工标注二十个查询的相关证据。验收时分别给出三类任务的错误，证明自己没有把几何接近误当成所有

任务的正确标签。

第三次实验，我们用小模型做一次 SFT，再做一次可控续训。先制作高质量讲题示范，再加入偏短的新课程答案，观察旧任务变化；从同一起点比较回放、训练强度和数据风格。可以使用 Transformer 强化学习工具库（Transformer Reinforcement Learning，TRL）、LLaMA-Factory 等工具；后者是项目名称，不必展开成另一套缩写，但要记录版本并按相应官方文档配置。验收是分项收益与退化的证据，不是训练命令跑完。

第四次实验，我们做微型 RFT。对一批训练题多次采样，用可靠验证器筛选，抽查过程，再训练一次。与相同预算的原始 SFT 基线比较，报告题目覆盖、通过率、训练 token 与留出评估结果。若有条件，再增加一个小规模 RLVR 实验；若没有，就用保存的奖励组分析零方差与探索瓶颈。

第五次实验，我们比较普通 SFT 与 DPO。用同一 SFT 起点和少量人工核对的偏好对，检查训练前后真实回答。特别加入长度相近但质量不同的样本，避免让长度替代质量。我们可以把 OPD 作为选做：只有在老师概率接口、tokenizer 对齐和预算都具备时才实施，否则先复核公开实验与估计器。

第六次实验，我们搭建最小 RAG，再接上计算器工具。保存每次检索与动作，加入无答案、资料冲突、工具超时等案例。最终测试使用未参与开发的新问题，并同时报告模型单独回答与完整系统回答。我们由此知道改进来自哪里，也知道系统仍在哪些地方会失败。

一个可行节奏是：前两周完成模型基础，第三周做表示，第四至五周做 SFT 与退化，第六至八周做采样、RFT、RL 诊断和偏好学习，最后两至四周完成推理与应用项目。我们按掌握程度调整，不把日历进度当作理解的证据。

停下来试一试从六次实验中选一个，写出最小可执行规模、固定变量、唯一改动、验收指标和可能失败原因。若资源减少一半，说明先删什么，以及哪些诊断必须保留。

本节相关资料

• Brown 等，2020，Language Models are Few-Shot Learners。原文链接 。对应基础模型与上下文学习实验。

• Ouyang 等，2022，InstructGPT。原文链接 。对应示范、反馈与独立评估的串联。

• Lewis 等，2020，Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks。原文链接 。对应最后的知识增强系统。

8.6　研究闭环从失败证据决定下一步

回顾前面的学习时，我们重新打开第一天的题。小助现在能解释方程，也能依据课程资料回答截止时间，遇到信息不足时说明缺口。它仍可能失败，但我们已经能把失败变成下一轮工作，而不是只说 “大模型有幻觉” 或 “再训练一下”。

如果它不知道一个新事实，我们先确认资料是否存在以及是否应该检索。如果它知道事实却不按要求表达，我们检查提示与示范。如果它偶尔成功却不稳定，我们测采样与选择，再考虑 RFT 或 RL。

如果奖励涨而真实表现不涨，我们审查目标和泛化。如果训练完全无效，我们从最基本的信号链路开始。

我们也重新理解了最初的婴儿类比。广泛接触、示范练习、结果反馈，确实能帮助我们记住训练阶段之间的联系；真正指导工程的，则是每次输入、监督、分布与更新方式。故事让知识有位置，机制让位置不会误导我们。

一份合格的结课报告，应让另一组同学无需听我们的口头解释就能判断结果。我们写明任务、数据来源与切分、模型与模板版本、训练或检索改动、评价方法和成本；展示总体与分项结果；保存失败案例和可复现配置；最后只提出证据能够支持的结论。没有提升的实验也有价值，只要它排除了一个合理假设。

我们给下一阶段留一个研究题：在同样预算下，针对小助的一类具体缺陷，哪一种反馈最有效？我们可以比较增加示范、拒绝采样、偏好对和老师逐 token 反馈，但必须先控制起点、覆盖与预算。若无法同时控制，就明确哪些因素仍然混在一起，避免把数据收益误写成算法胜利。

到这里，整个体系已经接上了：预训练形成可生成的分布；表示学习支持数据组织与检索；后训练改变任务行为；采样与反馈重新分配成功路径；推理时计算增加尝试或过程；RAG 与工具提供外部信息和动作；评估再把失败带回数据与方法。我们以后遇到新论文，就把它放进这个循环，问它实际修复了哪里。

停下来试一试不使用任何算法缩写，用一页文字复述“小助从第一天到现在”的经历。每出现一个新方法，都要先交代前一步暴露的具体问题。若某个概念找不到它要解决的问题，就回到对应小节补上上下文。

本节相关资料

- Ouyang 等，2022，InstructGPT。原文链接 。重读时画出数据流、监督流与评估流。
- Rafailov 等，2023，DPO。原文链接 。重读时说明它简化了哪些组件，保留了哪些假设。
- Agarwal 等，2023，GKD。原文链接 。重读时解释为什么训练前缀的来源值得单独研究。

第九章　可解释性入门

9.1　可解释性的证据看到了什么与证明了什么

前面的训练诊断让我们发现：小助的回答变短，可能只是行为倾向变化；一个词生成不出来，也可能涉及输出端偏移。我们已经能通过输入与输出比较排除一些解释，但还会追问：模型内部到底利用了哪些信息，又通过什么计算形成这次答案？这就是可解释性（interpretability）要帮助我们研究的问题。

我们先区分两个尺度。行为分析改变提示、数据或工具，观察系统如何反应；机制可解释性（mechanistic interpretability）进一步研究参数、激活与计算路径，尝试用可检验的机制解释特定行为。解释的对象可以是一道题、一个注意力头，也可以是跨一批输入反复出现的计算模式。我们必须先写明解释范围，不能从一个例子跳到“整个模型就是这样思考的”。

最容易上手的是注意力热图。假设小助生成“她”时，一个头对某个人名分配较大权重，这是一条有价值的观察。但权重只描述该头怎样混合 value；value 的内容、输出投影、残差中的其他信息和后续层的计算同样影响答案。高权重不能单独证明“这个词导致了回答”，低权重也不能保证“这个词毫无作用”。

小助还可以自己解释：“我先根据代词找到人名，再回答。”我们把这种文字当成另一个需要验证的输出。它可能忠实，也可能遗漏影响因素，或者合理化一个已经偏向的答案。可见推理过程有教学和检查价值，却不能直接替代内部因果证据。

我们把证据分成三层：观察到某个激活与行为相关；从表示中读出某类信息；干预某处计算后，目标行为按预测改变。后两层也有各自的假设和局限。接下来，我们先学习怎样“读出”，再学习怎样做受控干预。

停下来试一试对“模型注意到名字”“模型表示了人物关系”“这个头参与选择答案”三句话，分别写出需要什么证据。若三句都只拿同一张热图支持，我们还没有区分观察与因果。

本节相关资料

• Jain 与 Wallace，2019，Attention is not Explanation。原文链接 。阅读重点：注意力权重为什么不足以自动充当解释；不要反过来推成“注意力完全没有分析价值”。

• Turpin 等，2023，Language Models Don’t Always Say What They Think Unfaithful Explanations in Chain-of-Thought Prompting。原文链接 。阅读重点：解释文本如何遗漏实际影响回答的因素。

9.2　线性探针表示中能读出什么

我们怀疑小助在某一层已经表示了“谁把物品给了谁”，但最后没有选对人名。怎样判断信息是在中途丢失，还是存在却没有被后续计算有效利用？一种入门办法是在线性探针（linear probe）上做实验：冻结小助，只训练一个很小的读出器。

我们收集一批带标签的句子，取指定层、指定位置的隐藏向量 h，再训练线性分类器预测标签 c。这里的标签可以是语法属性或一个明确的关系类别。为了把表示变成类别概率，探针可采用如下形式：

$$P_\phi(c \mid h) = softmax(Wh + b)_c$$

这里 W 和 b 是探针的权重与偏置，φ 是它们的统称，下标 c 表示取第 c 类的概率。原语言模型参数保持冻结，只有探针更新；softmax 沿类别维度归一化。我们要测的是“这个小读出器能否利用 h 完成标签任务”，不是重新训练小助解决原问题。

如果探针在独立数据上表现很好，可以说明这些表示中存在可由该读出器利用的信息。它不能单独证明小助实际使用了这条信息。一个学生的草稿上写着正确公式，也可能在最后做题时没有用到它；探针与原模型后续计算同样不是一回事。

我们因此设置控制。训练与测试按句型和实体分组，防止只记住名字；与输入词频、位置等简单基线比较；用打乱标签检查是否只是高容量探针在记忆。我们尽量保持探针简单，并报告正则化与数据量。探针越复杂，越难区分“从原表示读取”与“读出器自己学会了任务”。

不同层的探针准确率可以帮助定位下一步实验，但不要把曲线峰值直接命名为“真正的知识层”。标签定义、读取位置与数据集变化都可能改变结论。我们会带着探针发现的候选位置，进入因果干预。

停下来试一试固定一个小模型，按句型划分训练与测试，比较同一个简单探针在几层上的表现。加入随机标签与只用位置的基线。写出探针成功时能支持的最强结论，以及它还不能支持的结论。

本节相关资料

• Alain 与 Bengio，2016，Understanding intermediate layers using linear classifier probes。原文链接 。阅读重点：冻结模型、训练读出器的实验结构，以及表示可读出性与原任务行为的区别。

• Wang 等，2022，Interpretability in the Wild A Circuit for Indirect Object Identification in GPT-2 small。原文链接 。阅读重点：为什么机制分析需要把表示观察与更直接的行为干预结合。

9.3　激活修补用干预检验计算路径

我们准备一个比课程问答更容易控制的小任务。提示是“When Alice and Bob went to the library, Alice gave the book to”，模型应更偏向续写 Bob。把后半句的 Alice 换成 Bob，预期答案随之换成 Alice。这类例子让我们研究间接宾语识别（Indirect Object Identification，IOI），避免一开始就试图解释开放式长回答。

首先，我们确认所选模型在这批原始提示上确实表现出目标行为，并检查候选名字在 tokenizer 中是否各占一个 token。若候选是多 token，就要比较完整续写的对数概率，不能只看首 token。这里先使用单 token 候选，方便定义测量。

我们把末尾位置对 Bob 和 Alice 的 logits 分别记为 z 的下标 Bob 与 Alice。为了避免只看最终采样是否碰巧成功，用分数差衡量模型倾向：

$$\Delta z = z_{Bob} - z_{Alice}$$

Δz 越大，表示相对更偏向 Bob；它是 logit 差，不是正确率，也不必在 0 到 1 之间。对照输入改变了答案角色，所以应先确认其基线分数差与原始输入确实不同。若没有可测量的行为差异，后续 “恢复” 实验就缺少对象。

接着做激活修补（activation patching）：缓存原始运行某层、某位置的激活；运行对照输入时，只把这一处替换为原始激活，其他计算保持对照运行的状态；然后观察 Δz 是否朝原始方向恢复。这是在固定参数下修改一次前向计算，和改写权重的模型编辑不同。

图 9.1 中，两次运行共享全部权重。竖向虚线只替换一个明确指定的中间状态；其后计算继续使用对照输入的其余状态。我们记录干预前后的 logit 差，而不是只挑一条采样成功的答案。

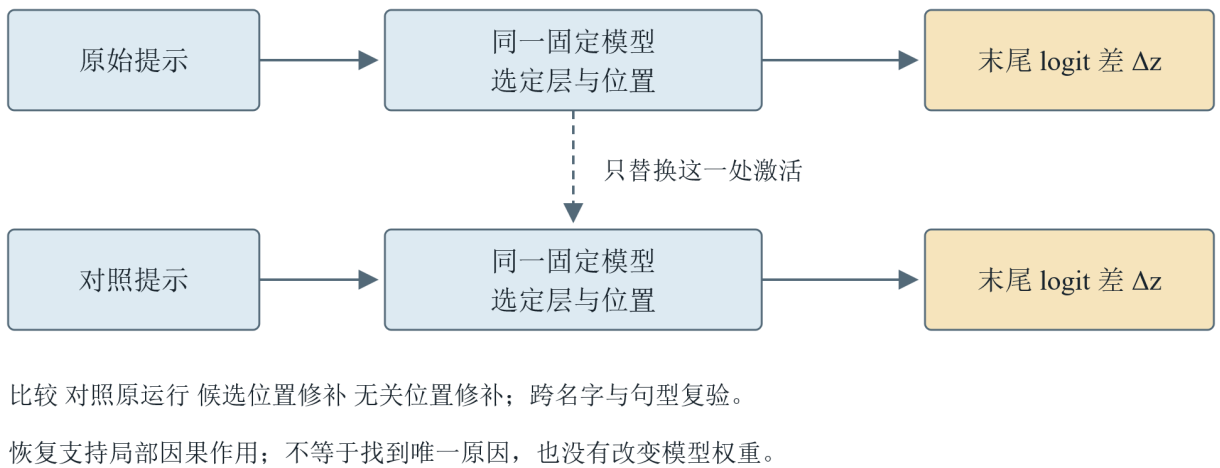


图 9.1 激活修补的受控实验。虚线是人为干预，不是模型自己跨提示传递的信息；无关位置修补是必要的负对照之一。

如果一个位置的修补稳定恢复了倾向，我们获得了 “在这组输入和干预方式下，这处状态携带了影响输出的信息” 的证据。它不表示这里存着全部知识，更不表示找到了唯一原因。我们还要换不同名字、句型与对照方式复验，并修补无关位置作负对照。把不相容的激活硬塞进去，也可能制造偏离正常分布的状态。

我们再用消融（ablation）检查另一侧：移除、替换为均值或重采样某个组件的输出，行为是否受损？不同消融方式会产生不同分布偏移；冗余路径存在时，一个有作用的组件也未必是单独必要的。把多处组件与信息流组织成可重复检验的计算回路（circuit），才比一张“重要位置热图”更接近机制说明。

停下来试一试对同一组提示比较不修补、候选位置修补、无关位置修补三种运行。保留每条提示的 Δz，而不只汇报最漂亮的一例。写清楚修补的是哪种激活、哪层、哪个 token 位置，以及对照输入怎样构造。

本节相关资料

• Wang 等，2022，Interpretability in the Wild A Circuit for Indirect Object Identification in GPT-2 small。原文链接 。阅读重点：怎样把一个可控行为分解成候选组件与计算路径。

• Zhang 与 Nanda，2023，Towards Best Practices of Activation Patching in Language Models Metrics and Methods。原文链接 。阅读重点：对照构造、度量和修补方式为什么会改变结论。

9.4　叠加表示与稀疏自编码器一个神经元未必对应一个概念

做完修补，我们查看一个候选神经元的高激活文本，发现它既回应课程说明，也回应某些代码片段。我们很想给它起名为“规则神经元”，但新的例子很快推翻了这个名字。问题可能在于：模型有用的特征并不总沿单个坐标轴排列。

叠加表示（superposition）研究一种可能性：许多特征利用不完全正交的方向共享有限维度。当每次只出现少量相关特征时，这种共享可能仍有用。它帮助我们理解为什么一个神经元可能响应多种模式，却不是所有神经元混杂行为的唯一解释，也不意味着任意向量方向都天然对应人类概念。

稀疏自编码器（Sparse Autoencoder，SAE）尝试把某层激活重新表示为一个更大的、但每次只激活少数项的特征向量。编码器把原激活 h 映射为特征 f(h)，解码器再重建出 h 的估计值。一个便于理解的训练目标，同时惩罚重建误差与特征激活的绝对值之和：

$$L_{SAE} = E_h \left[\| h - \hat{h}2 \|_2^{\|} + \lambda \| f(h) \|_1 \right]$$

这里 h 是收集到的模型激活，h 上的帽号是重建值，第一项是平方重建误差；第二项的 1 范数表示各特征激活绝对值之和，λ 控制稀疏程度。这只是入门的 L1 型目标，实际 SAE 还有其他稀疏机制，并需约束解码向量尺度，防止仅靠缩放降低惩罚。训练 SAE 时，原模型保持冻结。

如果若干特征更容易用一致的文本模式描述，我们获得了更便于分析的坐标系。可“高激活样本看起来都像同一主题”仍是标签假设。我们还要找反例、看新数据、检验重建质量，并检查干预该特征后行为是否按预测改变。稀有特征可能被漏掉，过多特征也可能把一个概念拆成多份。

我们特别避免一句诱人的结论：“已经找到诚实特征，所以调高它就能保证模型诚实。”特征干预可能影响多种行为，也可能只对局部输入有效。SAE 是产生机制假设的工具，不是天然的真值坐标系，

更不能替代前面建立的独立任务评估。

停下来试一试先读一个公开 SAE 的特征例子，检查它展示了哪些高激活文本、是否有反例、是否报告重建与行为干预。若只有一个形象标签，请把它改写为可验证的模式描述，暂时不要称其为“某概念的开关”。

本节相关资料与研究材料

• Elhage 等，2022，Toy Models of Superposition。原文链接 。阅读重点：在可控玩具模型里，稀疏性与特征共享如何相互影响；与真实模型的结论分开。

• Gao 等，2024，Scaling and evaluating sparse autoencoders。原文链接 。阅读重点：稀疏程度、重建质量与评估如何权衡。

9.5　最小解释实验从一个问题走到可复核的结论

我们把可解释性入门收束成一次小实验，目标不是给小助画一张包罗万象的“大脑地图”，而是解释一个窄行为：在某类人物关系句子上，哪些中间状态影响下一名字的选择？我们选用可访问中间激活的小型 Transformer，先确保基线行为足够稳定，再开始分析。

第一步建立一小批原始与对照提示，按句型分出开发组和未见测试组。记录 tokenizer、模型检查点、候选 token、位置对齐方式与基线 Δz。若原始输入就经常失败，我们先缩小任务或换更合适的设置，不能用解释技术代替行为基线。

第二步只选择一种激活类型，例如每层指定位置的残差流（residual stream，即各组件输出通过残差连接累加的表示），逐处缓存与修补。每次只改一个对象，保存输入、源激活、目标位置和输出分数。可视化各位置的恢复情况，是为了筛选候选，不是立即给每层赋予语义。

第三步固定筛出的少量候选，在未见名字与句型上复验，加入无关位置和相同输入自修补等控制。检查效果是否依赖一个特殊对照，是否牺牲其他行为。需要时再补线性探针或细分到注意力头；不要同时更换任务、模型、指标与干预方式。

我们可以把实验计划写成一张紧凑记录表：

环节	固定或记录的内容	通过条件
行为基线	原始与对照提示候选及 token 切分	目标倾向有可测量差异
候选筛选	激活类型层与位置修补来源	效果超过无关位置控制
独立复验	新实体新句型另一种对照	方向与预设假设一致
结论范围	模型版本任务与干预定义	不外推为通用概念开关

最后写报告时，我们把“事实观察”“机制假设”“未排除的解释”分开。一个清楚的结论可以是：“在该模型和这类句子上，把第某层某位置的原始残差激活移入对照运行，会使候选分数差向原始方向

移动。”这比“模型在这一层理解了人类关系”克制，但也更有内容，因为别人能够复查和推翻它。

我们现在可以把这一章接回训练诊断。内部分析能帮助提出新假设，比如输出头是否改变、某条纠错路径是否受损；要证明修复对真实用户有用，仍需回到独立题集、旧能力回归和成本评估。机制证据与行为证据相互补充，才让我们的解释既具体，也不超出实验所能支持的范围。

停下来试一试用一页实验记录写出问题、假设、原始与对照输入、干预、负对照、未见测试和结论边界。若无法获得模型内部激活，就先完成设计和行为基线，明确哪些因果问题暂时不能回答。

本节相关资料

• Zhang 与 Nanda，2023，Towards Best Practices of Activation Patching in Language Models Metrics and Methods。原文链接 。阅读重点：把输入构造与度量选择纳入实验报告。

• Wang 等，2022，Interpretability in the Wild A Circuit for Indirect Object Identification in GPT-2 small。原文链接 。阅读重点：从候选筛选走向可重复验证的回路假设。

附录术语与阅读方法

关键术语与边界

术语	我们在本书中的用法
基座模型	主要经过预训练的模型不表示完全无法遵循提示
预训练	用广泛数据建立表示与生成能力本书以因果语言建模为主
SFT	在给定输入与目标输出上做监督微调可以学习格式也可以学习任务内容
RFT	本书特指拒绝采样微调其他资料中需先确认全称
on-policy	数据与当前策略相关具体还要检查采样版本与更新滞后
off-policy	本书宽泛指使用外部或固定行为数据不等于方法必然无效
奖励	优化时使用的数值信号可能只是实际质量的代理
参考模型	提供相对概率或偏离约束的固定策略不等于 PPO 的旧采样策略
蒸馏	学生学习老师的输出或分布可以离线也可以在学生轨迹上进行
幻觉	生成中出现无依据或错误内容需要结合任务证据具体判定
RAG	先获取外部资料再条件生成不保证检索或生成永远正确
验证集	用于开发比较的数据反复用于选择后不再是独立最终证明